



ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse

Jongmin Kim^{*†}, Gwangho Lee^{*†}, Sangpyo Kim[†], Gina Sohn[†], Minsoo Rhu[‡], John Kim[‡], Jung Ho Ahn[†]

[†]Seoul National University, Seoul, South Korea

{jongmin.kim, g_corey, vnb987, gina.gemini, gajh}@snu.ac.kr

[‡]KAIST, Daejeon, South Korea

{mrhu, jjk12}@kaist.edu

Abstract—Homomorphic Encryption (HE) is one of the most promising post-quantum cryptographic schemes that enable privacy-preserving computation on servers. However, noise accumulates as we perform operations on HE-encrypted data, restricting the number of possible operations. Fully HE (FHE) removes this restriction by introducing the bootstrapping operation, which refreshes the data; however, FHE schemes are highly memory-bound. Bootstrapping, in particular, requires loading GBs of evaluation keys and plaintexts from off-chip memory, which makes FHE acceleration fundamentally bottlenecked by the off-chip memory bandwidth.

In this paper, we propose ARK, an Accelerator for FHE with Runtime data generation and inter-operation Key reuse. ARK enables practical FHE workloads with a novel algorithm-architecture co-design to accelerate bootstrapping. We first eliminate the off-chip memory bandwidth bottleneck through runtime data generation and inter-operation key reuse. This approach enables ARK to fully exploit on-chip memory by substantially reducing the size of the working set. On top of such algorithmic enhancements, we build ARK microarchitecture that minimizes on-chip data movement through an efficient, alternating data distribution policy based on the data access patterns and a streamlined dataflow organization of the tailored functional units—including base conversion, number-theoretic transform, and automorphism units. Overall, our co-design effectively handles the heavy computation and data movement overheads of FHE, drastically reducing the cost of HE operations, including bootstrapping.

Keywords—fully homomorphic encryption; domain-specific architecture; algorithm-architecture co-design

I. INTRODUCTION

Homomorphic encryption (HE) is a cryptographic method that enables third parties to process data blindly. HE is regarded as a game-changer in privacy-preserving cloud computing because HE does not leak information about the data to anyone other than the owner, allowing secure offloading of computation to untrusted servers.

HE schemes are based on the learning with errors [82] paradigm, a computational problem using noise for security. Noise accumulates as we continuously apply computation to encrypted data, which can eventually explode and make the data impossible to decrypt. Leveled HE (LHE) schemes

only allow a restricted number of operations (ops) to prevent noise explosion, thus only supporting simple workloads [7], [18]. An HE op called *bootstrapping* can be used to lower the noise level and allow an unbounded number of ops over encrypted data. Due to its significance, this class of HE schemes capable of bootstrapping is referred to as *fully HE* (FHE) [41]. Among numerous FHE schemes [17], [18], [29], [36], [39], CKKS [27], our primary target FHE scheme, is gaining popularity because it supports real arithmetic and is thus more suitable for machine learning (ML) services.

Despite its potential, however, the high memory and computational costs of HE are preventing its wide adoption. When data is encrypted using an HE scheme, the size expands by more than an order of magnitude, and an op as simple as multiplication transforms into a complex sequence of ops. Prior works have tried to mitigate such overhead by using CPU extensions [15], [56], GPUs [3]–[5], [55], FPGAs [60], [61], [83], [85], and ASICs [81], [87]. Recent proposals suggesting accelerated ASIC/FPGA designs, F1 [87] in particular, have proven successful for LHE schemes.

However, the parameters used in these designs were not large enough to effectively support CKKS bootstrapping. Bootstrapping is an expensive op, dominating the execution time of complex FHE workloads [55], [64], [65]. Moreover, FHE schemes supporting bootstrapping must use large parameters [16], [28], which correspondingly increase the data size. In fact, the working set of bootstrapping can reach several GBs, which must be fetched from off-chip memory due to limited on-chip memory capacity [37], [53]. Given such constraints of FHE schemes, previous hardware accelerators are severely bottlenecked by the low off-chip memory bandwidth.

In this paper, we present a set of algorithmic enhancements that address the off-chip memory bandwidth bottleneck of FHE workloads: *minimum key-switching* and *on-the-fly limb extension*. The key benefits of our proposed algorithms are twofold. First, our proposed algorithms drastically reduce the memory bottleneck of FHE, eliminating 88% of off-chip memory access in evaluating a major op composing bootstrapping. Second, our proposal enables the deployment

*Jongmin Kim and Gwangho Lee are co-first authors.

of massive computational logic and on-chip memory, large enough to hold the reduced working set on-chip, a feature lacking in conventional systems.

We design ARK, an FHE accelerator that effectively exploits the increased arithmetic intensity enabled by our proposed algorithmic enhancements, addressing new computational and data movement requirements arising from FHE support. ARK features novel functional units (FUs) tailored to the primary functions of FHE, including base conversion (BConv), number-theoretic transform (NTT), and automorphism. The FUs lower the bandwidth pressure on register files (RFs) and minimize data movement. For seamless operation across the FUs, ARK utilizes a data distribution policy that adapts to two data access patterns of the primary functions and simplifies the dataflow by splitting the chip into two logical regions.

By co-designing the algorithms and the architecture, ARK outperforms the state-of-the-art HE accelerator [87] in multiplicative throughput [55] by $2,353\times$ and in logistic regression training [43] by $18\times$. ARK also enables a real-time CNN inference on encrypted data; ARK performs an inference with the ResNet-20 model in 0.125 seconds, $18,214\times$ faster than the baseline CPU implementation [64]. ARK is sized 418.3mm^2 and consumes up to 281.3W of power.

In summary, our major contributions are as follows:

- We analyze CKKS bootstrapping in detail and identify the memory bottleneck in hardware acceleration of FHE induced by excessive single-use data during bootstrapping.
- We devise two algorithms to mitigate the memory bottleneck of FHE by substantially reducing the single-use data in bootstrapping. These algorithms break new ground specifically for domain-specific architectures by overcoming the performance bound imposed by the memory bottleneck.
- We design ARK, an algorithm-architecture co-designed accelerator for FHE. We deploy specialized FUs that minimize on-chip data movement on ARK. ARK adapts data and job distribution based on the data access patterns and fixes dataflow for streamlined execution across the chip.

II. BACKGROUND

We explain the pertinent details of HE with an emphasis on CKKS. We adopt notations and terminologies from [34], [44]. Parameters and notations are summarized in Table I.

A. Homomorphic encryption (HE)

HE is an encryption method that allows computation on encrypted data. It is based on a computational problem called ring learning with errors (RLWE), which is known to be difficult to solve even for quantum computers [6]. There are a variety of HE schemes that support different data types, such

as integers [17], [18], [39], boolean [29], [36], and fixed-point complex (real) numbers [27]. In particular, CKKS [27], our primary target HE scheme, supports the encryption of fixed-point complex vectors, making it suitable for numerous real-world applications such as machine learning.

In CKKS, a *message* is an unencrypted vector of length n . Each entry of a message vector is called a *slot*, which can contain a complex number. *Encryption* of a message consists of two steps. First, we transform a message $\mathbf{m}$ into a polynomial $P_{\mathbf{m}}$ in a cyclotomic polynomial ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$, which is a degree- $(N - 1)$ polynomial with coefficients in $\mathbb{Z}_Q = \{0, 1, \dots, Q - 1\}$, the set of integers modulo Q ; we treat a polynomial as a vector in $\mathbb{Z}_Q^N$ where the entries are the polynomial's coefficients. The maximum number of slots included in a single message $\mathbf{m}$ is determined by N as $n \leq N/2$. We perform this transformation by computing the Inverse Discrete Fourier Transform (IDFT) of $\mathbf{m}$, multiplying a scale factor Δ , and rounding the result:

$$P_{\mathbf{m}} \simeq \Delta \cdot \text{IDFT}(\mathbf{m}) \quad (1)$$

Then, $P_{\mathbf{m}}$ is converted into a *ciphertext* $[\mathbf{m}]$ based on Eq. 2. $A_{\mathbf{m}}$ is a randomly sampled polynomial, S is a secret key polynomial, and E is a small random error polynomial.

$$[\mathbf{m}] = (B_{\mathbf{m}}, A_{\mathbf{m}}), \quad B_{\mathbf{m}} = A_{\mathbf{m}} * S + P_{\mathbf{m}} + E \quad (2)$$

Decryption is the inverse of the encryption process. We first recover the polynomial $P_{\mathbf{m}}$ from $[\mathbf{m}]$ using the secret key S . Then, we apply DFT as shown in the following equation.

$$\mathbf{m} \simeq \Delta^{-1} \cdot \text{DFT}(P_{\mathbf{m}} + E) = \Delta^{-1} \cdot \text{DFT}(B_{\mathbf{m}} - A_{\mathbf{m}} * S) \quad (3)$$

Due to the random error polynomial E included in the encryption process, the decrypted result is not identical to $\mathbf{m}$. However, we can consider the decrypted result as $\mathbf{m}$ because E is a polynomial with small coefficients.

B. Computational optimizations for HE ops

HE schemes use a fixed set of computations called *HE ops* to process encrypted data without decryption. However, HE ops are expensive. A polynomial in R_Q is a long ($N = 2^{16}$) vector of large integers ($Q > 2^{1000}$). In general, two techniques are used to mitigate the cost of HE ops.

Number-Theoretic Transform (NTT): NTT is used to lower the complexity of a polynomial multiplication (mult). Multiplying two polynomials in R_Q is a (negacyclic) convolution of two length- N vectors and has an $\mathcal{O}(N^2)$ complexity. NTT is a variant of DFT, which converts the convolution into an element-wise mult, reducing the complexity to $\mathcal{O}(N)$:

$$\text{NTT}(P_1 * P_2) = \text{NTT}(P_1) \cdot \text{NTT}(P_2)$$

While NTT reduces the complexity of a polynomial mult, there is an additional cost of performing NTT. Thus,

Table I
CKKS PARAMETERS AND NOTATIONS

Params	Description
N	Degree. Length of a plaintext polynomial.
n	Length of a message. $n \leq N/2$.
Q	Polynomial modulus.
P	Special modulus.
$\mathcal{B}$	$\{p_0, p_1, \dots, p_{\alpha-1}\}$, a set of prime limbs of $P = \prod_{i=0}^{\alpha-1} p_i$.
$\mathcal{C}$	$\{q_0, q_1, \dots, q_L\}$, a set of prime limbs of $Q = \prod_{i=0}^L q_i$.
C_i	Partial limb group of $\mathcal{C}$. $C_i = \{q_{\alpha i}, q_{\alpha i+1}, \dots, q_{\alpha(i+1)-1}\}$, $i = 0, 1, \dots, \text{dnum}-1$.
$\mathcal{D}$	$\mathcal{B} \cup \mathcal{C}$, a set of prime limbs of PQ .
L	Maximum multiplicative level.
ℓ	(Current) multiplicative level. A level- ℓ polynomial has $\ell + 1$ limbs. $0 \leq \ell \leq L$.
dnum	Decomposition number.
α	$(L + 1)/\text{dnum}$. The number of prime limbs in C_i and $\mathcal{B}$.
Δ	Scale multiplied during encryption.
$\mathbf{m}$	Message, a vector of n slots.
P, P_m	Polynomial. Subscript $\mathbf{m}$ emphasizes that it corresponds to a message $\mathbf{m}$.
$[\mathbf{m}]$	Ciphertext encrypting a message $\mathbf{m}$.
$[P]_{q_i}$	q_i -limb of P .
$[P]_{\mathcal{C}}$	$\{[P]_{q_i} : q_i \in \mathcal{C}\}$, the set of q_i -limbs of P for $q_i \in \mathcal{C}$.
$\mathbf{evk}$	Evaluation key. An $\mathbf{evk}$ for $\text{HMult}(\mathbf{evk}_{\text{mult}})$ and $\mathbf{evks}$ for HRots with different rotation amount r 's ($\mathbf{evk}_{\text{rot}}^{(r)}$'s) exist.

Fast Fourier Transform (FFT) algorithms [32], [40] with $\mathcal{O}(N \log N)$ complexities are applied to compute NTT, and we keep polynomials in their NTT-applied versions to prevent unnecessary NTTs and Inverse NTTs (INTTs). We refer to this NTT-applied version as the *evaluation representation* of the polynomial, and we assume every polynomial is in its evaluation representation unless otherwise stated. However, as there are functions that cannot be performed when a polynomial is in its evaluation representation, we often perform INTT to bring back the polynomial to the original representation, which we refer to as the *coefficient representation*.

Residue Number System (RNS) and limbs: P_m has coefficients in $\mathbb{Z}_Q$ having up to 1,000s of bits. RNS is widely used to reduce the high complexity of performing multi-precision arithmetic between large integer coefficients [11], [25]. We first choose $(L + 1)$ word-sized prime numbers to form an ordered set $\mathcal{C} = \{q_0, \dots, q_L\}$ such that $Q = \prod_{i=0}^L q_i$; we refer to each q_i as a *prime limb* of Q . Based on the Chinese Remainder Theorem (CRT), any coefficient $c \in \mathbb{Z}_Q$ can be uniquely represented with a vector $(c \bmod q_0, \dots, c \bmod q_L)$, allowing a single polynomial $P_m \in \mathbb{R}_Q$ to be decomposed into $(L + 1)$ polynomials in $\{\mathbb{R}_{q_i}\}_{0 \leq i \leq L}$. Under such representation, we treat P_m as a matrix of size $(L + 1) \times N$ words and denote it as $[P_m]_{\mathcal{C}}$. We refer to each row of the matrix as the polynomial's *limb*. We use the term *q_i -limb* to indicate a specific row, for example, the i -th row. It is denoted as $[P_m]_{q_i}$ and is

Algorithm 1 BConvRoutine

Notation: P_{coeff} = coefficient representation of P

```

1: procedure BCONVRoutine( $[P]_{\mathcal{B}}, \mathcal{C}$ )
2:    $[P_{\text{coeff}}]_{\mathcal{B}} \leftarrow \text{INTT}([P]_{\mathcal{B}})$ 
3:    $[P_{\text{coeff}}]_{\mathcal{C}} \leftarrow \text{BConv}([P_{\text{coeff}}]_{\mathcal{B}})$ 
4:    $[P]_{\mathcal{C}} \leftarrow \text{NTT}([P_{\text{coeff}}]_{\mathcal{C}})$ 
5:   return  $[P]_{\mathcal{C}}$ 
6: end procedure

```

obtained by applying modulo q_i to the coefficients. Using RNS and NTT, a polynomial op becomes element-wise ops between $(L + 1) \times N$ matrices that only involve word-sized arithmetic.

Base conversion (BConv) [11] is used to change the prime limb set of a polynomial. To perform BConv, the polynomial must be in its coefficient representation. Thus, it is common in CKKS to perform a series of $\text{INTT} \rightarrow \text{BConv} \rightarrow \text{NTT}$, which we refer to as a *BConvRoutine* (Alg. 1). BConv converts $[P_{\text{coeff}}]_{\mathcal{B}}$ for a prime limb set $\mathcal{B} = \{p_0, p_1, \dots, p_{\alpha-1}\}$ into $[P_{\text{coeff}}]_{\mathcal{C}}$ by Eq. 4, where $\hat{p} = \prod_{i \neq j} p_i$ for $\forall p_i \in \mathcal{B}$.

$$\begin{aligned}
[P_{\text{coeff}}]_{\mathcal{C}} &= \text{BConv}_{\mathcal{B} \rightarrow \mathcal{C}}([P_{\text{coeff}}]_{\mathcal{B}}) \\
&= \left\{ \sum_{j=0}^{\alpha-1} ([P_{\text{coeff}}]_{p_j} \cdot \hat{p}_j^{-1} \bmod p_j) \cdot \hat{p}_j \bmod q_i \right\}_{0 \leq i \leq L} \quad (4)
\end{aligned}$$

We further explain the details of BConv in Section V-A.

C. Primitive HE ops in CKKS

In Table II, we summarize the *primitive* HE ops of CKKS. These HE ops become the building blocks of complex HE ops such as homomorphic linear transform. We describe some of the important details of the primitive HE ops below. **HRescale:** After multiplying (CMult, PMult, or HMult) a ciphertext with another operand having a scale of Δ , the scale of the result becomes Δ^2 . HRescale is a primitive HE op that divides the ciphertext by Δ to recover its scale back to Δ .

As it is difficult to perform a division when polynomials are represented in limbs using RNS, CKKS utilizes an alternative way to obtain the divided result. In CKKS, the value of each q_i is set close to Δ and HRescale is performed by eliminating the last limb of the polynomial (q_L -limb) and multiplying $q_L^{-1} \bmod q_i$ ($i = 0, 1, \dots, L - 1$) to the remaining limbs [25]. We can successively apply HRescale after each mult until the polynomial only has the q_0 -limb left. At this moment, it is impossible to perform additional mults. Thus, the maximum number of possible mults is L , which we refer to as the *maximum multiplicative level*. We call the remaining number of possible mults the *(current) multiplicative level* and denote it as ℓ . A ciphertext at a multiplicative level ℓ is represented as a pair of $(\ell + 1) \times N$ matrices.

Table II
PRIMITIVE HE OPS OF CKKS

Operation	Result	Description
CAdd($\llbracket \mathbf{m} \rrbracket, c$)	$\llbracket \mathbf{m} + \mathbf{c} \rrbracket = (\mathbf{B}_m + \mathbf{c}, \mathbf{A}_m)$	Add a scalar c to a ciphertext. $\mathbf{c}$ is a length- N vector with every entry c .
CMult($\llbracket \mathbf{m} \rrbracket, c$)	$\llbracket \mathbf{m} \cdot \mathbf{c} \rrbracket = (\mathbf{B}_m \cdot \mathbf{c}, \mathbf{A}_m \cdot \mathbf{c})$	Multiply a scalar to a ciphertext.
PAdd($\llbracket \mathbf{m} \rrbracket, \mathbf{P}_{m'}$)	$\llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (\mathbf{B}_m + \mathbf{P}_{m'}, \mathbf{A}_m)$	Add an unencrypted polynomial to a ciphertext.
PMult($\llbracket \mathbf{m} \rrbracket, \mathbf{P}_{m'}$)	$\llbracket \mathbf{m} \cdot \mathbf{m}' \rrbracket = (\mathbf{B}_m * \mathbf{P}_{m'}, \mathbf{A}_m * \mathbf{P}_{m'})$	Multiply an unencrypted polynomial to a ciphertext.
HAdd($\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket$)	$\llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (\mathbf{B}_m + \mathbf{B}_{m'}, \mathbf{A}_m + \mathbf{A}_{m'})$	Add two ciphertexts.
HMult($\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket, \mathbf{evk}_{\text{mult}}$)	$\llbracket \mathbf{m} \cdot \mathbf{m}' \rrbracket = \text{KeySwitch}(\mathbf{A}_m * \mathbf{A}_{m'}, \mathbf{evk}_{\text{mult}}) + (\mathbf{B}_m * \mathbf{B}_{m'}, \mathbf{A}_m * \mathbf{B}_{m'} + \mathbf{A}_{m'} * \mathbf{B}_m)$	Multiply two ciphertexts.
HRot($\llbracket \mathbf{m} \rrbracket, r, \mathbf{evk}_{\text{rot}}^{(r)}$)	$\llbracket \mathbf{m} \ll r \rrbracket = \text{KeySwitch}(\psi_r(\mathbf{A}_m), \mathbf{evk}_{\text{rot}}^{(r)}) + (\psi_r(\mathbf{B}_m), \mathbf{0})$	Perform a circular left shift by r slots. ψ_r is an automorphism performed on polynomials.
HRescale($\llbracket \mathbf{m} \rrbracket$)	$\llbracket \Delta^{-1} \cdot \mathbf{m} \rrbracket = (\Delta^{-1} \mathbf{B}_m, \Delta^{-1} \mathbf{A}_m)$	Restore the scale of a ciphertext having scale Δ^2 back to Δ .

Automorphism: To circularly shift a message by r slots through HRot, we first compute the automorphism of polynomials. Automorphism moves the i -th coefficient of a polynomial to the position of the $\psi_r(i)$ -th coefficient by applying the mapping in Eq. 5 to each limb. We reuse the notation to denote automorphism on a polynomial $\mathbf{P}_m$ as $\psi_r(\mathbf{P}_m)$ and refer to r as the *rotation amount*.

$$\psi_r : i \mapsto i \cdot 5^r \bmod N \quad (5)$$

Key-switching: HMult and HRot produce polynomials that are only decryptable with keys $\mathbf{S}^2$ or $\psi_r(\mathbf{S})$. To make such a polynomial decryptable with the secret key $\mathbf{S}$, we perform *key-switching* during HMult or HRot by multiplying it with a special public key called the *evaluation key* ($\mathbf{evk}$):

$$\text{KeySwitch}(\mathbf{P}_m, \mathbf{evk}) = P^{-1} \cdot \mathbf{P}_m \cdot \mathbf{evk} \quad (6)$$

In particular, the $\mathbf{evk}$ encrypting $\mathbf{S}^2$ is used for HMult and is denoted as $\mathbf{evk}_{\text{mult}}$. The $\mathbf{evk}$ encrypting $\psi_r(\mathbf{S})$ is used for HRot and is denoted as $\mathbf{evk}_{\text{rot}}^{(r)}$. A separate $\mathbf{evk}_{\text{rot}}^{(r)}$ is required for each rotation amount r .

In general, key-switching is an expensive HE op as it involves multiple NTTs and BConvs. Moreover, because key-switching is performed frequently, it dominates the execution time of an HE application [55]. Multiplying $\mathbf{P}_m$ and $\mathbf{evk}$ requires additional steps as they have different modulus. While $\mathbf{P}_m \in R_Q$ has modulus Q , $\mathbf{evk} \in R_{PQ}$ has modulus PQ where P is the *special modulus* that can be decomposed into a set of α prime limbs $\mathcal{B} = \{p_0, p_1, \dots, p_{\alpha-1}\}$. BConvs are performed to bring $\mathbf{P}_m$ into R_{PQ} by generating the $[\mathbf{P}_m]_{\mathcal{B}}$ part. After multiplying $\mathbf{P}_m$ with $\mathbf{evk}$, BConvs bring the multiplied results back to R_Q , and the results are divided by P to reduce error in a similar way to HRescale.

Security of HE and generalized key-switching: We use the generalized key-switching method in [44]. It introduces a new parameter, dnum , and sets the number of special prime limbs as $\alpha = (L+1)/\text{dnum}$. A single $\mathbf{evk}$ consists of dnum pairs of polynomials: $\mathbf{evk} = (\mathbf{evk}_0, \mathbf{evk}_1, \dots, \mathbf{evk}_{\text{dnum}-1})$, $\mathbf{evk}_i \in R_{PQ}^2$. Also, the prime limb set $\mathcal{C}$ is decomposed into dnum *partial limb groups*: $\mathcal{C}_i = \{q_{\alpha i}, q_{\alpha i+1}, \dots, q_{\alpha(i+1)-1}\}$

Algorithm 2 Generalized key-switching

Notation: $\mathbf{evk} = \{\mathbf{evk}_i\}_{0 \leq i < \text{dnum}}$, $\mathbf{P} = \{[\mathbf{P}]_{\mathcal{C}_i}\}_{0 \leq i < \text{dnum}}$

```

1: procedure KEYSWITCH( $\mathbf{P}, \mathbf{evk}$ )
2:   for  $i \leftarrow 0, 1, \dots, \text{dnum}-1$  do
3:      $[\mathbf{P}]_{\mathcal{D}} \leftarrow [\mathbf{P}]_{\mathcal{C}_i} \cup \text{BConvRoutine}([\mathbf{P}]_{\mathcal{C}_i}, \mathcal{D} \setminus \mathcal{C}_i)$ 
4:   end for
5:    $(\mathbf{B}, \mathbf{A}) \leftarrow \sum_{i=0}^{\text{dnum}-1} \text{PMult}(\mathbf{evk}_i, [\mathbf{P}]_{\mathcal{D}})$ 
6:    $\mathbf{B}' \leftarrow [\mathbf{B}]_{\mathcal{C}} - \text{BConvRoutine}([\mathbf{B}]_{\mathcal{B}}, \mathcal{C})$ 
7:    $\mathbf{A}' \leftarrow [\mathbf{A}]_{\mathcal{C}} - \text{BConvRoutine}([\mathbf{A}]_{\mathcal{B}}, \mathcal{C})$ 
8:   return  $(P^{-1} \cdot \mathbf{B}', P^{-1} \cdot \mathbf{A}')$ 
9: end procedure

```

($0 \leq i < \text{dnum}$). $(L+1)$ limbs of the polynomial $\mathbf{P}$ are decomposed into dnum pieces as well: $\{[\mathbf{P}]_{\mathcal{C}_i}\}_{0 \leq i < \text{dnum}}$. Before multiplying each $\mathbf{evk}_i$ with the corresponding piece $[\mathbf{P}]_{\mathcal{C}_i}$, we perform BConvRoutine on each piece $[\mathbf{P}]_{\mathcal{C}_i}$ to extend the limbs to R_{PQ} (line 3 in Alg. 2). Then, we multiply dnum pairs of $\mathbf{evk}_i$ s each with $[\mathbf{P}]_{\mathcal{D}}$ and accumulate them (line 5). BConvRoutines bring the result back to R_Q (line 6-8).

The computational complexity of key-switching and the size of an $\mathbf{evk}$ increase as dnum gets larger, degrading overall performance. Despite such limitations, generalized key-switching is used to acquire more multiplicative levels. The *security level* of HE is roughly decided by N and $(\alpha + L + 1)$, which increases as N gets larger or $(\alpha + L + 1)$ gets smaller [33]. Recent HE studies [16], [66], [68] and libraries [38], [79] typically target a security level of 128 bits. Under such security level constraints, N is chosen as low as possible and $(\alpha + L + 1)$ as high as possible. Table III shows exemplar parameter selections satisfying 128-bit security. If dnum is large, the value of $\alpha = (L+1)/\text{dnum}$ reduces, enabling a higher L under the same $(\alpha + L + 1)$. In particular, when using a small N , it is inevitable to use a dnum value close to its maximum ($= L+1$) to secure as many multiplicative levels as possible.

D. CKKS bootstrapping

When the ciphertext $\llbracket \mathbf{m} \rrbracket$'s multiplicative level ℓ reaches 0 after multiple HRescales, it becomes impossible to perform

Table III
REPRESENTATIVE PARAMETERS USED IN HE ACCELERATION WORKS
AND CORRESPONDING DATA SIZES

Work	Parameters					Data size (MB)		
	N	L	L_{boot}	dnum	α	P_m	$\llbracket m \rrbracket$	evk
Lattigo [38]	2^{16}	24	15	5	5	12.5	25	150
100x [55] [†]	2^{17}	29	19	3	10	30	60	240
F1 [87]	2^{14}	15	-	16	1	1	2	34
ARK	2^{16}	23	15	4	6	12	24	120

[†] [55] used a 173-bit secure parameter set, while others 128-bit.

more mult ops. As such, one must recover the multiplicative level through a special op called *bootstrapping* to enable further ops on $\llbracket m \rrbracket$. Because bootstrapping allows an unlimited number of ops, it becomes an essential op for complex HE applications. The HE schemes that support bootstrapping are generally referred to as *Fully HE* (FHE).

Bootstrapping is a complex sequence of HE ops that consists of four major steps: LevelRecover, Homomorphic IDFT (H-IDFT), EvalMod, and Homomorphic DFT (H-DFT). During LevelRecover, we raise $\llbracket m \rrbracket$'s multiplicative level to its maximum, L , by changing the modulus from q_0 to Q . However, if we decrypt it using the secret key S , we get $P_{m'}$ instead of P_m due to the multiple of q_0 , which was previously removed through modular reduction (i.e., $P_{m'} = P_m + q_0 \cdot I$). To remove the $q_0 \cdot I$ term from $P_{m'}$, we first apply H-IDFT on $\llbracket m' \rrbracket$ to enable homomorphic ops on $P_{m'}$ instead of m' . Based on Eq. 1, we denote the resulting $\llbracket \text{IDFT}(m') \rrbracket$ as $\llbracket P_{m'} \rrbracket$ to emphasize that the ciphertext encrypts the coefficients of $P_{m'}$. EvalMod homomorphically performs the modulo operation on $\llbracket P_{m'} \rrbracket$. The modulo operation is not a linear function, so it is approximated by a high-degree polynomial function. Homomorphic evaluation of the function consists of multiple HMults and CMults, consuming many multiplicative levels [26]. This step recovers $\llbracket P_m \rrbracket$ by calculating $\llbracket P_{m'} \bmod q_0 \rrbracket$ and H-DFT recovers $\llbracket m \rrbracket$ from $\llbracket P_m \rrbracket$.

The resulting ciphertext has $(L - L_{boot})$ multiplicative levels as multiple HRescales consume L_{boot} levels during bootstrapping. Thus, L must be high enough to support CKKS bootstrapping. However, the security limit requires that L can only be increased proportionally to N , so N should be sufficiently high ($\geq 2^{15}$) to allow bootstrapping [68]. Prior works on FHE [16], [38], [55], [68] mostly use $N = 2^{15}$ to 2^{17} to preserve sufficient multiplicative levels for other ops. Thus, we use the parameter set summarized in Table III in this work, which is a slightly modified version of Lattigo [38]'s parameter set and guarantees 128-bit security.

III. FHE MEMORY BOTTLENECKS

A. Limitations of prior HE accelerators

All HE ops can be broken down into *primary functions* that can be executed in parallel: (I)NTT, BConv, automor-

phism, and other *element-wise functions*. By using well-known FFT algorithms [32], (I)NTT can be effectively parallelized by dividing it into $\log N$ stages, where each stage consists of parallel $N/2$ butterfly ops. Since N is as large as 2^{16} , the degree of parallelism is high. Given such, domain-specific architectures that employ a massive number of functional units (FUs) to reap such opportunities become an appealing option for accelerating HE. To this end, designing a memory system capable of feeding these abundant FUs on-chip becomes a primary design objective.

Prior work has identified that HE ops have very low arithmetic intensity and are bounded by the off-chip memory bandwidth [34], [55], [85]. Even a simple HE op, such as HMult, has a working set including several ciphertexts and an evk, which amounts to several hundreds of MBs. Unfortunately, conventional CPU/GPU/FPGA solutions typically do not come with on-chip memory large enough to hold the entire working set of HE ops (e.g., even the latest NVIDIA H100 GPU “only” has 50MB of on-chip cache [37]), causing frequent off-chip memory accesses and high memory bandwidth pressure.

F1 [87] is the first ASIC HE accelerator to evaluate the performance of a single-slot CKKS bootstrapping. F1 uses small parameters, $(N, L) = (2^{14}, 15)$, and deploys a massive amount of computation logic to utilize the parallelism in HE ops. In particular, F1 utilizes a dedicated *NTT unit* (NTTU) that implements a pipelined 2D-FFT using $\frac{1}{2}\sqrt{N}\log N = 896$ modular multipliers. F1 consists of 16 vector *clusters*, each with $\sqrt{N} = 128$ lanes that feed data to an NTTU and other FUs. Overall, F1 has a massive amount of 18,432 modular multipliers in a chip: 14,336 for NTTUs and 4,096 for element-wise multipliers. Every HE op is essentially a sequence of modular mults and modular adds, and a modular mult requires a costly modular reduction op [13], [71]. Thus, the computational capability of an HE accelerator can be quantified by the number of modular multipliers.

Although F1 provides high speedup compared to conventional systems, its applicability is limited to small problem sizes and simple workloads; when it comes to practical FHE parameters and complex workloads that require frequent bootstrapping, the performance of F1 can be degraded significantly.¹ We first explain the details of bootstrapping for analysis.

B. State-of-the-art bootstrapping algorithm

H-(I)DFT is a memory-intensive op that takes up most of the execution time in bootstrapping. H-(I)DFT can be performed in a similar pattern to FFT as shown in Alg. 3,

¹Because of the small parameters used in F1, F1 cannot perform bootstrapping of ciphertexts encrypting a message vector, only supporting a message with a single slot ($n = 1$). Thus, F1's bootstrapping throughput is severely limited. Single-slot bootstrapping does not need H-(I)DFT steps, so it is less costly and consumes fewer levels. F1 also uses 32-bit machine word and prime sizes, but practical bootstrapping requires the use of larger primes for error mitigation.

Algorithm 3 FFT-like homomorphic DFT (Radix- 2^k)

Require: Precomputed plaintexts (polynomials) $\{P_{s,i}\}$

```
1: procedure DFT( $\llbracket \mathbf{m} \rrbracket$ ,  $\{P_{s,i}\}$ )
2:    $\llbracket \mathbf{t} \rrbracket \leftarrow \llbracket \mathbf{m} \rrbracket$ 
3:   for  $s \leftarrow 0, 1, \dots, \log_{2^k} n - 1$  do
4:      $\llbracket \mathbf{t} \rrbracket \leftarrow \sum_{i=-2^{k+1}}^{2^k-1} \text{PMult}(\text{HRot}(\llbracket \mathbf{t} \rrbracket, i \cdot 2^{k \cdot s}), P_{s,i})$ 
5:      $\llbracket \mathbf{t} \rrbracket \leftarrow \text{HRescale}(\llbracket \mathbf{t} \rrbracket)$   $\triangleright$  consumes one level
6:   end for
7:   return  $\llbracket \mathbf{t} \rrbracket$ 
8: end procedure
```

where 2^k denotes the radix in FFT [32]. Each iteration requires $(2^{k+1} - 1)$ HRots and $(2^{k+1} - 1)$ PMults, each with a different $\text{evk}_{\text{rot}}^{(r)}$ and a different *plaintext*, a polynomial for precomputed DFT constants ($P_{s,i}$). The whole process consumes $\log_{2^k} n$ multiplicative levels, so the choice of k creates a trade-off between computation and level consumption.

Line 4 of Alg. 3 can be further improved by first performing the *pre-rotation* in Eq. 7,

$$\llbracket \mathbf{t}' \rrbracket = \text{HRot}(\llbracket \mathbf{t} \rrbracket, -2^k \cdot 2^{k \cdot s}) \quad (7)$$

then applying the BSGS (Baby-Step, Giant-Step) algorithm [26] as shown in Eq. 8. k_1 and k_2 in Eq. 8 are integers satisfying $k + 1 = k_1 + k_2$. Consequently, the number of HRots reduces from $\mathcal{O}(2^k)$ to $\mathcal{O}(\sqrt{2^k})$.

$$\underbrace{\sum_{j=0}^{2^{k_2}-1} \text{HRot}\left(\underbrace{\sum_{i=0}^{2^{k_1}-1} \text{PMult}(\text{HRot}(\llbracket \mathbf{t}' \rrbracket, i \cdot 2^{k \cdot s}), P'_{s,i,j}), j \cdot 2^{k_1+k \cdot s}}_{\text{baby-step}}\right)}_{\text{giant-step}} \quad (8)$$

In ARK with $n = 2^{15}$, we use $k = 5$, which consumes 3 levels, and (k_1, k_2) in Eq. 8 as $(3, 3)$. With additional optimizations, 40 HRots and 158 PMults are performed for each H-(I)DFT, and 40 distinct $\text{evk}_{\text{rot}}^{(r)}$ s and 158 plaintexts must be prepared, which amount to 6.4GB (resp., 0.6GB) of data for H-IDFT (H-DFT) (see Table III).

For the entire bootstrapping algorithm, which we use as the baseline algorithm throughout this paper, we adopted the implementation from [44] while applying parts of the optimizations from [16], [22], [68]. Refer to the papers for a more detailed explanation of the bootstrapping implementation.

C. Memory bottleneck in bootstrapping

The time to load $\text{evk}_{\text{rot}}^{(r)}$ s and plaintexts, which are used once per H-(I)DFT, becomes the hard bound of latency for H-(I)DFT. Even if we assume that integrating hundreds of MBs of on-chip memory is feasible with the current fabrication technology [1], [62], [80], [91], we must fetch the single-use data of H-(I)DFT from off-chip memory due to its large aggregate size and limited data reuse opportunities.

With the latest HBM3 [52], a system having a 3TB/s off-chip memory bandwidth is feasible [37], which would allow loading the single-use data in 2.1ms (resp., 0.2ms) for H-IDFT (H-DFT).

We analyze the impact of such hard latency bound on domain-specific architectures by measuring the maximum utilization of FUs in F1. We scaled F1 to the bootstrappable parameter we use (see Table III) by utilizing NTTUs that feature $\frac{1}{2}\sqrt{N} \log N = 2,048$ modular multipliers to fully support bootstrapping. As a result, the total number of modular multipliers in the chip increases by $2.22\times$ to 40,960. We first computed the number of possible modular mults in the scaled F1 in 2.1ms (resp., 0.2ms) by assuming that it runs at 1GHz and is fully-pipelined so that 40,960 modular mults are possible every cycle. Then, we divided it by the number of modular mults performed in H-IDFT (H-DFT) to derive the maximum achievable utilization of modular multipliers. Overall, our analysis revealed that the scaled F1 performs poorly when handling off-chip memory bandwidth bottleneck caused by bootstrapping, only achieving an 8.61% (13.32%) utilization rate for H-IDFT (H-DFT).

Resolving the off-chip memory bandwidth bottleneck in bootstrapping should precede improving the computational capability for practical FHE workloads that require frequent bootstrapping, especially for domain-specific architectures that can utilize a massive amount of computational logic.

IV. ALGORITHMS TACKLING MEMORY BOTTLENECK

With the advancement of fabrication technology, the scaling of off-die memory bandwidth is slower than that of logic density [69]. We present our fabrication-technology-aware algorithmic enhancements that reduce off-chip memory access, alleviating FHE's memory bandwidth bottleneck.

A. Minimum key-switching (Min-KS)

[42] proposed *minimal key-switching* strategy for H-(I)DFT, which can be applied to the two following computational patterns found in H-(I)DFT, Eq. 9 corresponding to the baby-step and Eq. 10 to the giant-step.

$$\left\{ \text{HRot}\left(\llbracket \mathbf{x} \rrbracket, i \cdot r, \text{evk}_{\text{rot}}^{(i \cdot r)}\right) \right\}_{0 \leq i \leq m} \quad (9)$$

$$\sum_{i=0}^m \text{HRot}\left(\llbracket \mathbf{x}_i \rrbracket, i \cdot r, \text{evk}_{\text{rot}}^{(i \cdot r)}\right) \quad (10)$$

Eq. 9 involves rotating the same ciphertext, and Eq. 10 involves rotating and accumulating different ciphertexts, each with rotation amounts showing an *arithmetic progression* ($i \cdot r$). Both computational patterns must load m different evk s ($\text{evk}_{\text{rot}}^{(r)}, \text{evk}_{\text{rot}}^{(2r)}, \dots, \text{evk}_{\text{rot}}^{(m \cdot r)}$) from off-chip memory (see Fig. 1(a)), which makes HE workloads bottlenecked by the off-chip memory bandwidth. [42] instead computes each pattern iteratively, while reusing previous HRot computation

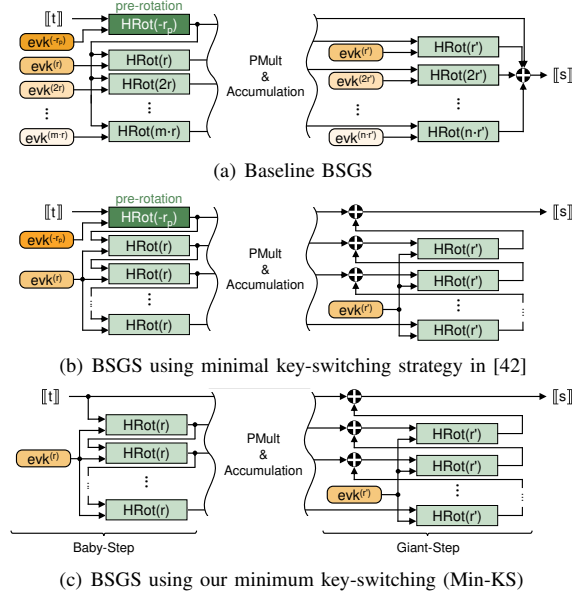


Figure 1. Computational flow of the BSGS algorithm in H-(I)DFT (a) with the baseline algorithm (Eq. 8), (b) with minimal key-switching strategy in [42], and (c) with our minimum key-switching (Min-KS).

results as in Eq. 11. Then, all HRots have the same rotation amount r , and consequently use the same single $\text{evk}_{\text{rot}}^{(r)}$.

$$\text{HRot}(\llbracket \mathbf{x} \rrbracket, i \cdot r) = \text{HRot}(\underbrace{\text{HRot}(\llbracket \mathbf{x} \rrbracket, (i-1) \cdot r)}_{\text{previous result}}, r) \quad (11)$$

Therefore, only three evks are required for BSGS computation when using the strategy (see Fig. 1(b)), one each for the pre-rotation (Eq. 7), the baby-step, and the giant step.

We build on the strategy and propose *minimum key-switching* (Min-KS), which further reduces evk requirements and has a more broad scope of application. First, we eliminate the pre-rotation by adjusting and canceling out the rotation amounts between the H-(I)DFT iterations in Alg. 3. Then, we modify the computational flow of BSGS (see Fig. 1(c)), through which it requires only two evks per H-(I)DFT iteration.

Second, we generalize Min-KS and apply it to similar computational patterns found in HE workloads. HE ops such as homomorphic convolution require multiple HRots with rotation amounts exhibiting an arithmetic progression; those are the possible targets of Min-KS. Also, we aggressively modify the data organization and algorithms to force the rotation amounts to show an arithmetic progression and apply Min-KS to HE ops, such as slot accumulation.

B. On-the-fly limb extension (OF-Limb)

We propose *on-the-fly limb extension* (OF-Limb), which generates the limbs of plaintexts used in PMult and PAdd on-the-fly to save off-chip memory access. The plaintext $\mathbf{P}_{\mathbf{m}'}$ used in PMult($\llbracket \mathbf{m} \rrbracket, \mathbf{P}_{\mathbf{m}'}$) is precomputed before executing

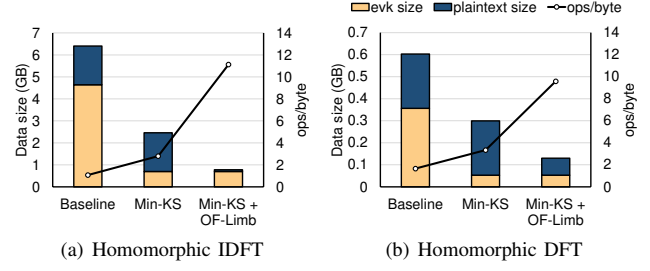


Figure 2. Data loaded from off-chip memory and arithmetic intensity (ops/byte) change when applying Min-KS and OF-Limb algorithms to (a) homomorphic IDFT and (b) homomorphic DFT.

an HE application in the same way as in Eq. 1. Precomputed $\mathbf{P}_{\mathbf{m}'}$ has the same number of limbs ($\ell+1$) as the polynomials composing $\llbracket \mathbf{m} \rrbracket$ and is loaded from off-chip memory for PMult or PAdd. However, we identify that it is possible to precompute only the q_0 -limb of $\mathbf{P}_{\mathbf{m}'}$ and extend it on-the-fly using the following equation so that $\mathbf{P}_{\mathbf{m}'}$ has $(\ell+1)$ limbs:

$$[\mathbf{P}_{\mathbf{m}'}]_{\mathcal{C}} = \{\text{NTT}([\mathbf{P}_{\mathbf{m}'}]_{q_0} \bmod q_i)\}_{q_i \in \mathcal{C}} \quad (12)$$

OF-Limb reduces off-chip memory access of PMult and PAdd to $1/(\ell+1)$ of the original method. When assuming that the evks and plaintexts are loaded from off-chip memory and the rest of the working set fits into on-chip memory, plaintexts take up 27.5% (resp., 40.9%) of off-chip memory access of H-IDFT (H-DFT).

OF-Limb incurs extra computation. NTT needs to be performed on the extended limbs of a plaintext to convert them into their evaluation representation. The increased computation accounts for 22.9% (resp., 24.1%) of the entire H-IDFT (H-DFT) computation. However, we identify that the performance gain from decreased off-chip memory access outweighs the overhead (in Section VII-B).

C. Impact of the optimizations on the arithmetic intensity of homomorphic (I)DFT

The two algorithms significantly reduce off-chip memory access, which eventually increases the arithmetic intensity (ops/byte) of H-(I)DFT. We computed the arithmetic intensity of H-(I)DFT of the baseline algorithm (Section III-B) with and without applying the two optimizations. The arithmetic intensity is defined as the number of modular mult ops divided by the off-chip memory access bytes. Min-KS results in a $2.6\times$ (resp., $2.0\times$) increase in the arithmetic intensity of H-IDFT (H-DFT), and OF-Limb results in an additional $4.0\times$ ($2.9\times$) increase in arithmetic intensity, which reaches 11.1 (9.6) ops/byte. The algorithms remove 88% (78%) of off-chip memory access, as shown in Fig. 2.

Other optimizations lower the compute cost of H-(I)DFT, such as hoisting [16], a technique to merge repeated computation, but they do not reduce the single-use data. As they are not compatible with Min-KS and only further reduce the

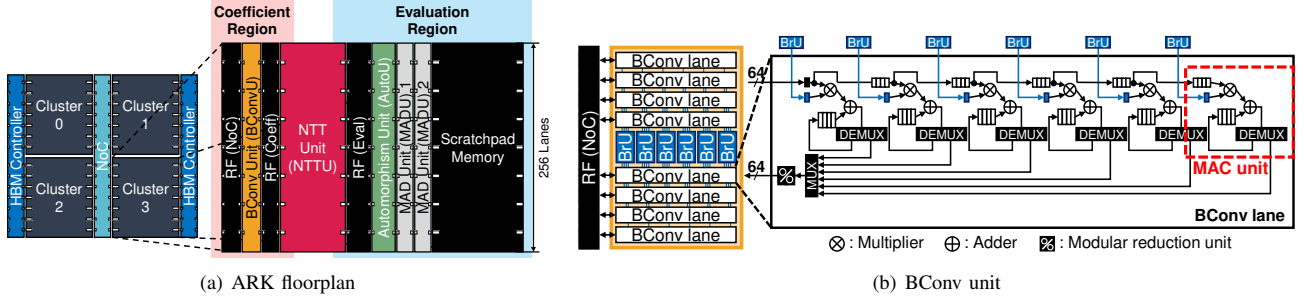


Figure 3. (a) Overview of ARK. ARK is composed of four clusters. A cluster includes a BConv unit (BConvU), an NTT unit (NTTU), an automorphism unit (AutoU), two multiply-add units (MADUs), scratchpad memory, and distributed register files (RFs). A cluster in ARK can be divided into two regions: the coefficient region and the evaluation region. (b) Organization of a BConv unit (BConvU). Eight BConv lanes are shown for simplicity. Each lane has six multiply-accumulate (MAC) units. Central broadcast units (BrUs) hold base tables.

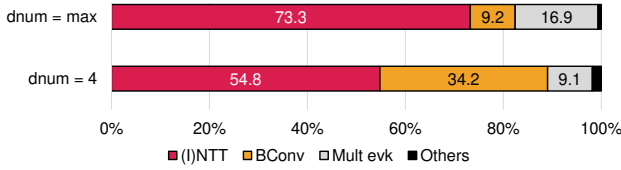


Figure 4. Computational breakdown, in terms of modular mult, of HRot with a ciphertext at the max level for different dnums. $(N, L) = (2^{16}, 23)$.

arithmetic intensity of the baseline H-(I)DFT algorithm, we excluded them from the analysis.

Min-KS and OF-Limb do not improve performance on conventional systems, such as CPUs and GPUs. On conventional systems, *evks* are always read from the off-chip memory due to the large size of *evks*; *evks* mostly do not fit in the on-chip storage of conventional systems. Therefore, reusing *evks*, which is enabled by Min-KS, has little impact on the execution time. Also, conventional systems lack the computational capability to handle the increased arithmetic intensity from OF-Limb cost-effectively. By contrast, for domain-specific architectures, we can design a system with enough memory capacity and computational capabilities to fully leverage these algorithms.

V. ARK ARCHITECTURE

We propose ARK, an architecture-algorithm co-design for FHE that can manage the increased arithmetic intensity and capitalize on the reduced working set by applying the two algorithms. ARK utilizes 512MB of on-chip scratchpad memory, enough to hold a couple of *evks* and temporary data of HE ops. Scaling in fabrication technology [21], [90] allowed such massive amounts of on-chip memory; recent domain-specific architectures [1], [62], [80], [91], including FHE accelerators [59], [88], deployed hundreds of MBs of on-chip memory. ARK extensively prefetches *evks*, which are often reused by Min-KS. Also, for the high NTT computation throughput required by OF-Limb, we leverage a high-throughput NTTU similar to that of F1 [87]. The NTTU

works on a vector of $\sqrt{N}$ elements every cycle, so ARK also has a form of a vector processor having $\sqrt{N} = 256$ lanes. Fig. 3(a) shows an overview of ARK with four clusters, each having an NTTU and other FUs for primary functions of CKKS: a BConv unit (BConvU), an automorphism unit (AutoU), and two multiply-add units (MADUs).

For practical bootstrappable parameters, F1's design targeting the maximum dnum is no longer applicable. When using max dnum = 24 under $(N, L) = (2^{16}, 23)$, for instance, the size of an *evk* reaches 600MB, which hinders data reuse as it becomes impossible to fit a single *evk* into on-chip memory. Moving to practical parameters using smaller dnums (see Table III) introduces new data access patterns and computational requirements. Thus, we design ARK to handle these new challenges in an FHE accelerator for bootstrappable parameters while using Min-KS and OF-Limb to relieve the off-chip memory bottleneck.

A. Specialized BConv unit for low dnums

Computational characteristics of HE ops change for lower dnums. Prior FPGA and ASIC HE accelerators [83], [85], [87] targeted small problem sizes with a low N , making it inevitable to use the max dnum to secure more multiplicative levels. When using the max dnum, (I)NTT takes up 73.3% of the total computation (Fig. 4), so accelerating (I)NTT could directly improve the performance. Therefore, most prior FPGA and ASIC HE accelerators focused on accelerating (I)NTT. However, for our practical bootstrappable parameters with a large N and a small dnum, the portion of (I)NTT drops to 54.8%, and BConv takes up 34.2% of the total computation.

We design a specialized BConv unit (BConvU) to handle increased BConv computation and relieve the register file (RF) pressure of BConv. BConv consists of two steps of modular mult. BConv first multiplies each p_j -limb by $(\hat{p}_j^{-1} \bmod p_j)$. Then, for each q_i , BConv multiplies each p_j -limb by $(\hat{p}_j \bmod q_i)$, and accumulates the results (see Eq. 4). BConvU targets the second step, which accounts for 96% of the computation of BConv. The second step is essentially a

matrix mult between an $(\ell + 1) \times \alpha$ matrix composed of $(\hat{p}_j \bmod q_i)$ s, which we refer to as a *base table*, and an $\alpha \times N$ matrix $[P_{\text{coeff}}]_{\mathcal{B}}$. A naïve matrix mult algorithm would require accessing $[P_{\text{coeff}}]_{\mathcal{B}}$ along the column direction while $[P_{\text{coeff}}]_{\mathcal{B}}$ is stored in memory in row-major order. To access the matrix along the row direction, our BConvU uses a block matrix mult algorithm [14], where each of multiple *BConv lanes* processes a column of the polynomial in parallel.

We utilize multiple modular mult-accumulate (MAC) units per lane, forming an output-stationary systolic array [54], [86] to reduce RF pressure. Fig. 3(b) depicts a Bconv lane composed of six MAC units organized as a 1×6 systolic array. A MAC unit passes the input coefficient of a polynomial to the next MAC unit to reuse input data. A MAC unit is entirely in charge of computing an inner product between a row in a base table and a column in a polynomial. On each cycle, each of the 256 MAC units on the same horizontal position receives a data element. These 256 data elements are from the same row of the polynomial, which are multiplied by the same value from the base table.

BConvU features central *broadcast units* (BrUs) that broadcast the base table elements to the MAC units. In each MAC unit, we time-multiplex mults between a row of the base table and four columns of the polynomial. This reduces the broadcasting frequency of BrUs to once every four cycles. We sweep over the number of MAC units to deploy in a lane and choose six, which provides sufficient throughput (see Section VII-C). By using six MAC units in a BConv lane, the entire BConv process boils down to partitioning the base table into $6 \times \alpha$ -shaped blocks and the polynomial into $\alpha \times (4 \cdot 2^8)$ -shaped blocks and performing the block matrix mult between the two.

Each MAC unit has a multiplier, an input buffer to hold four elements, and an output buffer to hold four accumulated results. Modular reduction logic is placed on the write-back path and is shared by the six MAC units. Instead, the output buffer of each MAC unit should hold accumulated results in 128 bits.

B. Access-pattern-oriented data distribution

BConv has a different data access pattern compared to other functions. Because an NTTU requires an entire limb of a polynomial to compute using a deep pipeline, and multiple (I)NTTs can be performed on separate limbs of a polynomial, it is natural to distribute a polynomial to multiple clusters *limb-wise*, which is also the data distribution policy F1 used. However, the limb-wise distribution becomes problematic for performing BConv, where multiple limbs of a polynomial are required to produce an output. This was not the case when using max dnum because the input set of limbs was composed of only a single limb when performing BConv, corresponding to $\mathcal{C}_i = \{q_{\alpha i}\}$ or $\mathcal{B} = \{p_0\}$ in Alg. 2.

To perform BConv in parallel while distributing data evenly among the clusters, we utilize a *coefficient-wise* data

distribution, where N coefficients of a limb in the coefficient representation are distributed evenly across the clusters. As BConv can be performed in parallel across the coefficients by splitting the $\alpha \times N$ matrix (polynomial) into $\alpha \times \frac{N}{4}$ matrices distributed across the four clusters, coefficient-wise distribution is suitable for the data access pattern of BConv.

We devise a data distribution policy that switches between limb-wise and coefficient-wise distribution and corresponding dataflow for performing HE ops in ARK. We focus on a BConvRoutine (Alg. 1). A polynomial limb passes through an NTTU that executes INTT. Each cluster prepares an INTT-applied limb and performs an all-to-all data exchange through the network-on-chip (NoC), switching to the coefficient-wise distribution. When each cluster holds a quarter of α limbs of a polynomial, BConv is executed by a BConvU. The reverse process is carried out for NTT. Other than BConv, limb-wise distribution is used.

Switching between limb-wise and coefficient-wise distribution requires all-to-all data communication between clusters. $(\alpha + L + 1) \cdot N$ words need to be transferred for every BConvRoutine, requiring a total of $(\text{dnum} + 2) \cdot (\alpha + L + 1) \cdot N$ words to be transferred in a single key-switching (see Alg. 2). Meanwhile, a limb-wise-only distribution incurs more data communication. For comparison, we implement the limb-wise-only data distribution by assigning $[P_{\mathbf{m}}]_{\mathcal{C}_i}$ to cluster i ($i = 0, 1, \dots, \text{dnum} - 1$) to eliminate data communication for BConvRoutine in Alg. 2. This implementation enables the clusters to perform the loop in line 2 of Alg. 2 independently. However, we need to redistribute the data for accumulation (line 5), which causes more data communication involving $2 \cdot \text{dnum} \cdot (\alpha + L + 1) \cdot N$ words when $\text{dnum} > 2$. We compare the performance of two data distribution policies in Section VII-C.

C. Two regions in a cluster and NTT unit

Our NTTU, depicted in Fig. 5, has unique components that focus on minimizing data movement and RF pressure. First, we distribute RFs across a cluster and organize the NTTU to have the opposite dataflows for INTT and NTT. In Fig. 5, data flows rightward for NTT and leftward for INTT. We partition the RF [8], [50], [84] and place RFs at both ends of the NTTU: RF (Coeff) and RF (Eval). Then, an NTTU splits a cluster into two regions: the *coefficient region* holding polynomials in the coefficient representation and the *evaluation region* holding polynomials in the evaluation representation. BConvs and data exchanges on the NoC are executed on the coefficient region, and the other functions are executed on the evaluation region. Such bidirectional dataflow simplifies the floorplan, which comes down to the organization shown in Fig 3(a), and enables a power-efficient design by reducing data movement that induces expensive wire traversal [46], [48]. RF (NoC) is added to communicate with the NoC and the BConvU. Revisiting the dataflow of a BConvRoutine, it can be observed from Fig. 3(a) that data

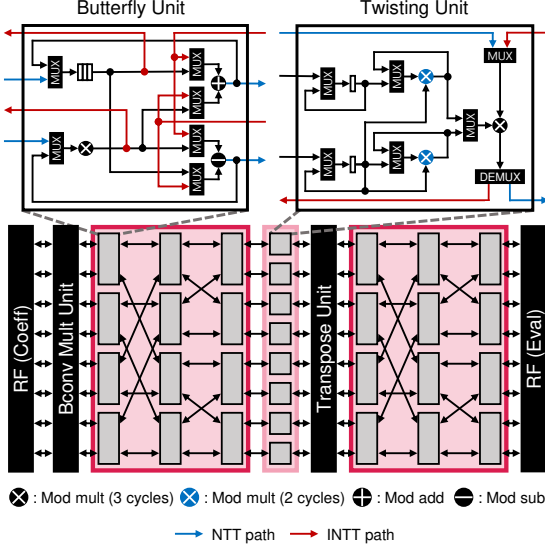


Figure 5. Organization of an NTT unit (NTTU). The logic for 64-point (I)NTT is shown for simplicity.

movement is minimized by flowing through RF (Eval) $\rightarrow$ NTTU $\rightarrow$ RF (Coeff) $\rightarrow$ NoC $\rightarrow$ RF (NoC) $\rightarrow$ BConvU. For INTT, NTTU also performs the first step of BConv using the *BConv mult unit* in Fig. 5.

Moreover, we devise *on-the-fly twisting factor generation* (OF-Twist) on our NTTU, which nearly eliminates memory traffic for loading *twisting factors*. F1 uses the 4-step FFT in [10], which performs N -point (I)NTT using 2D-FFT by first performing $\sqrt{N}$ -point (I)NTTs, multiplying elements with *twisting factors*, transposing the $\sqrt{N} \times \sqrt{N}$ matrix of elements, and finally performing additional $\sqrt{N}$ -point (I)NTTs. F1 implements the entire process as a single long pipeline by utilizing multiple butterfly units, twisting units, and a transpose unit. The benefit of this implementation is that the twiddle factors multiplied in the butterfly units stay the same for the whole process. By contrast, different twisting factors are multiplied for every element, so N twisting factors need to be loaded, of which the amount is as large as the actual data to perform NTT. We observe that the twisting factors show *geometric progression*. Thus, we design the twisting units to generate twisting factors on-the-fly by multiplying the common ratios of the geometric progression inside. Using OF-Twist, we only have to load the starting values and the common ratios, reducing the total data loaded during (I)NTT to roughly half. Also, OF-Twist can reduce the storage usage for twisting factors by 99%, saving 30MB of space ($2 \cdot (\alpha + L + 1) \cdot N$ words) on our parameter set (Table III).

D. Automorphism unit (AutoU)

Automorphism moves data according to the mapping ψ_r in Eq. 5. Automorphism may look like an irregular permutation between all N coefficients, but it can be performed

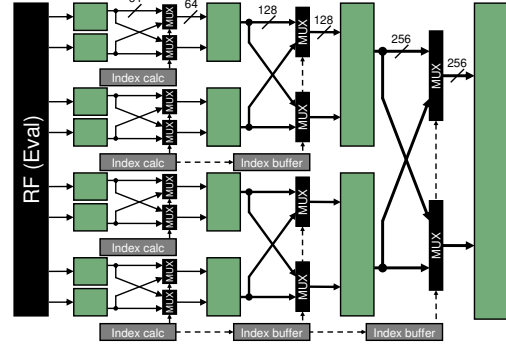


Figure 6. Organization of an automorphism unit (AutoU). The logic for eight words is shown for simplicity. The datapath is shown in solid lines and the control path is shown in dotted lines.

through internal permutations between 2^8 coefficients. Each 2^8 vector lane consumes a coefficient every cycle, where the 2^8 consumed coefficients have a stride of 2^8 ; i.e., the coefficients with indices $(i, i + 1 \cdot 2^8, i + 2 \cdot 2^8, \dots, i + (2^8 - 1) \cdot 2^8)$ are consumed every cycle ($0 \leq i < 2^8$). After automorphism, the 2^8 coefficients are again mapped to a set of 2^8 coefficients with a stride of 2^8 ; i.e., $(j, j + 1 \cdot 2^8, j + 2 \cdot 2^8, \dots, j + (2^8 - 1) \cdot 2^8)$ for $j = \psi_r(i) \bmod 2^8$. Thus, automorphism can be done by (i) loading 2^8 words (one from each lane), (ii) performing an internal permutation of 2^8 words according to ψ_r , and (iii) storing 2^8 words (one to each lane).

We devise an automorphism unit (AutoU)² implementing the internal permutation logic (see Fig. 6). An AutoU is composed of eight stages, recursively permuting the data according to the ψ_r index calculation results. Each stage is pipelined and merges two chunks of data by swapping the order of the two chunks depending on the index calculation results. The index calculation circuit includes a 16-bit multiplier multiplying the index with precomputed $5^r \bmod N$ (see Eq. 5), which can be computed with very little cost.

VI. IMPLEMENTATION

Hardware modeling: We modeled the FUs of ARK using ASAP7 [31], a predictive process design kit for a 7nm technology node. We used FinCACTI [89] to model the scratchpad and RFs as SRAMs because an SRAM compiler is not included in ASAP7. We updated the model of FinCACTI to match the information provided by ASAP7, the IRDS roadmap [49], and other published works on 7nm technology nodes [9], [20], [21], [53], [54], [75], [90], [94]. We separately modeled the overhead of expensive semi-global wires (80nm pitch [72]), where their length cannot be ignored. We used [54], [78] to evaluate the HBM memory. The peak power and area costs of ARK’s components are

²Our AutoU actually targets a polynomial in the evaluation representation, ψ_r , and the data address mapping change for polynomials in the evaluation representation, but the same property still holds.

Table IV
AREA AND PEAK POWER CONSUMPTION OF ARK.

Component	Area (mm ²)	Peak power (W)
4 BConvUs	9.3	18.9
4 NTTUs	57.2	95.2
4 AutoUs	20.6	4.6
8 MADUs	8.9	24.7
Register files	42.8	25.1
Scratchpad memory	229.2	54.0
NoC	20.6	27.0
HBM	29.6	31.8
Sum	418.3	281.3

shown in Table IV. Each peak power value is estimated with a usage case where the component can be most active. ARK is sized 418.3mm² and consumes up to 281.3W of power.

Performance modeling: We implemented a cycle-accurate simulator of ARK to evaluate its performance. The simulator takes an HE program written at a high level as an input and converts it to a data dependence graph of primary HE functions. The simulator schedules the functions and data movements while checking the data dependence and structural hazards. Static scheduling and software-controlled prefetching are possible as HE programs do not feature any dynamic control flows [87]. In that sense, our simulator acts as a VLIW compiler for ARK. The simulator collects the utilization rates of the components, combined with the power model, to derive power consumption.

FU implementation and wiring overhead: All FUs are fully-pipelined and run at 1GHz. In NTTUs and BConvUs, logic units for modular reduction use Montgomery reduction [71], and in MADUs, Barrett reduction [13]. Wiring overhead accounts for a significant portion of the FUs in area and power, particularly for NTTUs and AutoUs featuring long wires connecting different lanes in a cluster. The logic for 4 NTTUs (resp., 4 AutoUs) only accounts for 34.9mm² (0.9mm²) of area and 39.6W (0.3W) of power. The rest is attributed to the wiring overhead, including registers for the 1GHz operation of the wires and the repeaters [12]. The wiring overhead of NTTUs and AutoUs alone accounts for 10% of the area and 21% of the peak power of the entire ARK chip.

Memory and NoC implementation: Two 500GB/s HBM2 stacks [51], [52] are used, providing a total of 1TB/s off-chip memory bandwidth. On-chip memory components are single-ported, multiple-banked SRAMs. The scratchpad memory (128MB per cluster) runs at 1.25GHz and provides a bandwidth of 20TB/s chip-wide. RF (Coeff) runs double-pumped at 2GHz, while other RFs run at 1GHz. Distributed RFs (together 19MB per cluster) provide a bandwidth of 72TB/s chip-wide. NoC is implemented as a simple multiplexer network connecting the same lane from each cluster and provides a modest bandwidth of 8TB/s at 1GHz.

VII. EVALUATION

A. Experimental setup

We first compared the performance of ARK with other works using a metric called *amortized mult time per slot* [55] ($\mathbf{T}_{A.S.}$). [55] proposed $\mathbf{T}_{A.S.}$ as an important metric that allows fair comparison as a proxy compensating for the impact of parameter choice and reported multiplicative throughput in terms of $\mathbf{T}_{A.S.}$. $\mathbf{T}_{A.S.}$ adds HMult time for $1, \dots, L - L_{boot}$ levels ($\mathbf{T}_{mult}(\ell)$) with the bootstrapping time ($\mathbf{T}_{boot}$), then divides it by $L - L_{boot}$ and the number of slots refreshed in bootstrapping (n):

$$\mathbf{T}_{A.S.} = \frac{\mathbf{T}_{boot} + \sum_{\ell=1}^{L-L_{boot}} \mathbf{T}_{mult}(\ell)}{L - L_{boot}} \cdot \frac{1}{n} \quad (13)$$

We also evaluated ARK using FHE workloads that require bootstrapping. We did not include tiny LHE workloads as in [87], which do not require bootstrapping, as they have little practicality and are not targets of ARK. HELR [43] is a simple machine learning workload that trains a binary logistic regression classifier model for the MNIST dataset. In each iteration, it trains the model with a mini-batch containing 1,024 14×14-pixel images. We measured the performance of HELR for 30 iterations and reported the average execution time per iteration. We also measured the performance of ARK for more complex workloads. We performed CNN inference using Lee et al. [64]’s implementation of the ResNet-20 model for the CIFAR-10 [63] dataset. The model shows 91.31% accuracy. In [64], homomorphic evaluation of convolution layers in the model involves a series of HRots and PMults with weight plaintexts, so we applied both Min-KS and OF-Limb. Finally, we performed sorting [47]. Sorting in HE requires a complex evaluation of non-linear functions approximated by high-degree polynomials. OF-Limb was applied to every PMult in each workload.

For comparison, we used the state-of-the-art CKKS implementations of **Lattigo** [38], **100x** [55] and **F1** [87], each representing a single-thread CPU, GPU (NVIDIA V100 [77]) and ASIC implementation. We used the reported execution times in [55], [87], except for HELR, where [87] only reported the execution time for a single iteration with 256 images. We estimated the time for **F1** to perform 30 iterations of training with 1,024 images by assuming that **F1** trains 1,024 images over 4 iterations and performs $14 \times 14 = 196$ times of single-slot bootstrapping after each iteration while ignoring other additional costs in favor of **F1**. We also estimated execution times for a scaled-up version of **F1**, **F1+**, derived by optimistic compensation for the area and fabrication technology gap [75] between **F1** and ARK for a fair comparison. A similar methodology was used in [59]. We measured the performance of **Lattigo** by running the workloads on an Intel CPU system, Xeon Platinum 8358, with 512GB of DDR4-3200 memory.

Table V
T_{A.S.} AND HELR EXECUTION TIME OF ARK AND PRIOR WORKS

	Lattigo	100x	F1	F1+	ARK	vs. 100x
T _{A.S.} (μ s)	88	8	260	34	0.014	563×
HELR (ms)	23,293	775	1,024	132	7.421	104×

Table VI
PERFORMANCE OF ARK FOR RESNET-20 INFERENCE AND SORTING COMPARED TO CPU IMPLEMENTATIONS [47], [64]

	CPU	ARK	Speedup
ResNet-20 (s)	2,271 [64]	0.125	18,214×
Sorting (s)	23,066 [47]	1.990	11,590×

B. Performance of ARK

ARK outperforms all other prior works in terms of T_{A.S.}. ARK shows 563× lower (better) T_{A.S.} than 100x, which performs the best among the prior works (Table V). ARK shows 6,142× and 2,353× lower T_{A.S.} than Lattigo and F1+, respectively. F1 uses single-slot ($n = 1$) bootstrapping for CKKS which is severely limited in throughput, so it shows a similar level of T_{A.S.} to CPU (Lattigo).

ARK also outperforms prior works in HELR. ARK shows 3,139×, 104×, and 18× better performance than Lattigo, 100x, and F1+, respectively. The performance gap between ARK and other works decreases compared to T_{A.S.} due to reduced slot usage ($n = 256$) in bootstrapping for HELR. HE operations are comparable to SIMD operations involving all the slots. Therefore, a fundamental problem in HE is that, for small workloads like HELR that cannot utilize all the slots, throughput is greatly degraded. Bootstrapping in HELR uses only 256 out of 32,768 slots in ARK, thus is unable to extract the full capability of ARK. This is not the case for F1, which only supports single-slot bootstrapping, in turn showing much enhanced relative performance in HELR compared to other works. However, for more practical workloads, such as ImageNet [35] with 150,528 pixels in an image, such throughput degradation will be resolved.

ARK drastically reduces the execution time for complex workloads: ResNet-20 [64] and sorting [47] (Table VI). We used the reported performance of CPU implementation in each work as the baseline. ARK shows 18,214× and 11,590× speedup compared to the baselines. In particular, ARK performs a real-time CNN inference in 0.125s.

Impacts of algorithmic optimizations: Fig. 7(a) shows that our algorithms are effective in reducing the execution time of bootstrapping. Min-KS improves the performance of H-IDFT (resp., H-DFT) by 2.61× (1.43×). OF-Limb further improves the performance by 1.29× (1.04×), resulting in an aggregate performance improvement of 3.36× (1.48×). Overall 2.36× speedup is achieved in bootstrapping. Applying Min-KS resulted in a performance gain similar in ratio to the increase in ops/byte (see Fig. 2), whereas the performance improvement by additionally applying OF-

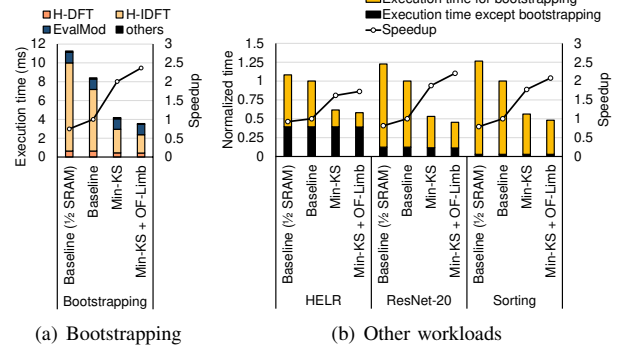


Figure 7. Execution time and speedup of (a) bootstrapping (single time) and (b) other workloads (normalized) while applying our algorithms incrementally. Execution time and slowdown of ARK when both algorithms are not applied and also the total scratchpad memory size is halved to 256MB (Baseline (1/2 SRAM)) are shown as well.

Limb falls short of the increase in ops/byte due to the increased computation.

Our algorithms also result in 1.72×, 2.20×, and 2.08× speedup for HELR, ResNet-20, and sorting, respectively, as shown in Fig. 7(b). Reduced slot usage ($n = 256$) in HELR leads to smaller m values in Eq. 9 and Eq. 10, so the overall benefit of Min-KS from reusing evks is reduced. Combined with the lower portion of bootstrapping (39.3%) in HELR, a relatively moderate speedup is achieved in HELR.

The speedup of ResNet-20 attributes partially to the speedup in the convolution layers of ResNet-20. We additionally applied Min-KS and OF-Limb to the convolution layers, which resulted in a 1.09× speedup of the execution time, excluding bootstrapping time. However, for HELR and sorting, only OF-Limb was applied besides bootstrapping, showing a minor performance improvement of less than 1%. Other than bootstrapping, these workloads do not feature a computation pattern where Min-KS is applicable.

Simply increasing the on-chip memory capacity has limited performance improvement without algorithmic enhancements. Fig. 7 shows that, when Min-KS and OF-Limb are not applied, doubling the total scratchpad memory size from 256MB to 512MB results in 1.34×, 1.08×, 1.23×, and 1.26× speedup for bootstrapping, HELR, ResNet-20, and sorting, respectively. By contrast, when the two algorithms are applied, the same doubling results in much higher 1.83×, 1.18×, 1.45×, and 1.50× speedup (parts of the results from Fig. 9(c) and Fig. 9(d)).

C. Alternative designs of ARK

We measured the performance and power of alternative designs of ARK to assess how each factor of ARK affects the performance and power. The results are shown in Fig. 8. The baseline ARK consumes power ranging from 100W to 135W, 44% of the peak power in geometric mean (gmean).

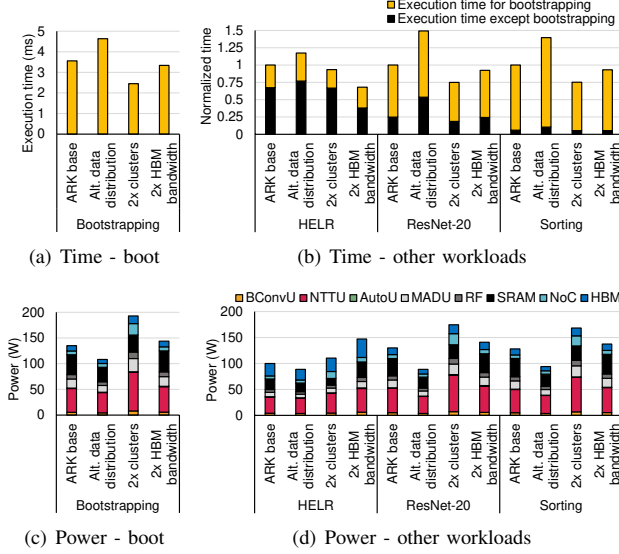


Figure 8. Execution time and average power for (a)(c) bootstrapping and (b)(d) other workloads with alternative designs of ARK: ARK using only limb-wise distribution (Alt. data distribution), eight-clustered ARK (2 $\times$ clusters), and ARK with 2TB/s main memory bandwidth (2 $\times$ HBM bandwidth). Normalized execution time is reported in (b).

Alternative data distribution: Using limb-wise-only distribution (see Section V-B) causes an overall performance drop. In the limb-wise-only implementation, the data transfer for accumulation after multiplying the evaluation key is the only data transfer. We designed an NoC suited for the pattern of this data transfer, which deploys adders inside the NoC to support the on-transit accumulation of data, preventing data concentration. However, even with this additional optimization, performance in bootstrapping, HELR, ResNet-20, and sorting degrades by 0.77 $\times$, 0.85 $\times$, 0.67 $\times$, and 0.72 $\times$, respectively. Power consumption decreases to 0.77 $\times$ of the baseline in gmean as the components become less busy.

More clusters: Because ARK has eliminated the off-chip memory bandwidth bottleneck, populating more computational logic leads to better performance. Here we doubled the number of clusters to eight, leading to twice aggregate scratchpad and RF bandwidth while fixing the total scratchpad size to 512MB. The eight-clustered design can perform bootstrapping 1.45 $\times$ faster and thus shows 1.33 $\times$ improved performance both for ResNet-20 and sorting. Due to the lower portion of bootstrapping in execution time, performance improvement in HELR is relatively modest with 1.07 $\times$ enhancement. However, this performance improvement comes at the cost of 1.29 $\times$ more power consumption on average and 1.39 $\times$ larger chip area. In particular, NoC power increases by 2.71 $\times$ on average. When considering the energy-delay-area product (EDAP) [92] to evaluate efficiency, the eight-clustered design shows 1.08 $\times$ higher EDAP, suggesting that the baseline ARK is more efficient.

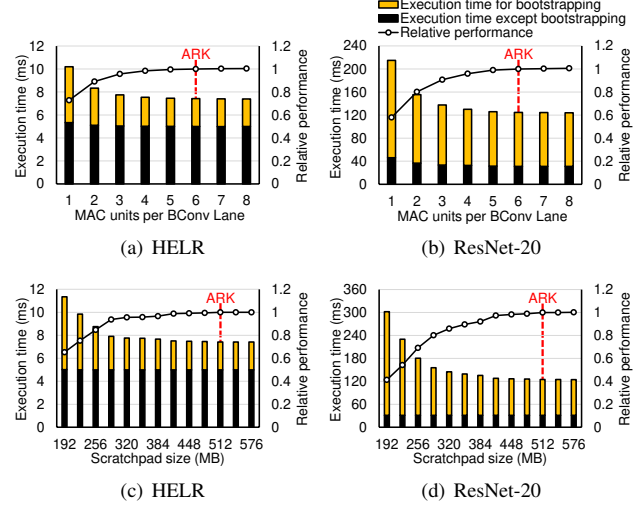


Figure 9. Performance of ARK for HELR and ResNet-20 with (a)(b) different numbers of MAC units per BConv lane and (c)(d) different sizes of total scratchpad memory.

Doubling HBM bandwidth: Because the proposed Min-KS and OF-Limb greatly diminish the off-chip memory bandwidth bottleneck, doubling the off-chip memory bandwidth to 2TB/s has little impact on the performance of bootstrapping, which improves by 1.07 $\times$. ResNet-20 and sorting, the performance of which highly depends on bootstrapping, also show a similar degree of performance improvements by 1.08 $\times$ and 1.07 $\times$, respectively. By contrast, HELR has memory-bound parts where multiple HRots with different $\mathbf{evk}_{\text{rot}}^{(r)}$ s must be performed. Min-KS is not applicable to those parts as their rotation amounts do not show arithmetic progression. Therefore, the design with doubled HBM bandwidth shows 1.47 $\times$ better performance for HELR.

Fig. 9 shows how performance is affected by the number of MAC units per BConv lane and the size of total on-chip scratchpad memory in HELR and ResNet-20.

Organization of a BConv lane: Increase in the number of MAC units in a BConv lane from one to six leads to 1.37 $\times$ and 1.72 $\times$ higher performance in HELR and ResNet-20, respectively, but after six, the performance saturates with less than 1% enhancement. HELR is less affected by the number of MAC units as the memory-bound parts of HELR, where multiple data with poor reusability must be loaded, are not accelerated with more computational capability.

Scratchpad memory capacity: As the amount of total on-chip scratchpad memory increases from 192MB to 512MB, the performance enhances by 1.53 $\times$ and 2.42 $\times$ in HELR and ResNet-20, respectively, and saturates. Bootstrapping time is very sensitive to the scratchpad memory size, but other HE ops, being performed at lower multiplicative levels than bootstrapping, need much smaller working sets and are barely accelerated with more scratchpad memory sizes.

VIII. RECENT FHE ACCELERATORS AND PRACTICALITY

Other FHE accelerators have been recently proposed along with ARK. Comparisons with those works under 128-bit secure settings are shown in Table VII. CraterLake [88] is a successor of F1 featuring 256MB of on-chip memory, massive BConv units, and evk generators which halve the memory traffic for evks. CraterLake uses coefficient-wise data distribution. ARK performs $1.23\times$ to $2.58\times$ faster in $T_{A.S.}$, HELR, and ResNet-20 compared to CraterLake. For HELR, [88] only reported performance for training the model with 256 images per iteration, so we estimated the time for CraterLake training 1,024 images over four iterations. [88] also reported the performance of CraterLake with 350MB of on-chip memory, which shows a speedup of $1.5\times$ in bootstrapping, but less than 10% for other workloads.

BTS [59] has a tiled architecture composed of 2,048 processing elements (PEs) that communicate through NoCs in the center. BTS performs an extensive search on optimal HE parameter sets for FHE accelerators under the observation that the loading time of evks becomes the performance bottleneck. Then, BTS optimizes the dataflow for HE ops so that they finish within the loading time. BTS features 512MB of on-chip memory and uses coefficient-wise data distribution. ARK performs $3.19\times$ to $15.32\times$ faster in $T_{A.S.}$, HELR, ResNet-20, and sorting compared to BTS.

Although huge chip designs are often found in GPUs [30], [37], [77] and domain-specific architectures [1], [62], [80], [91], such as Graphcore [62] with 896MB of on-chip memory and 823mm² of chip area, FHE accelerators might need to draw more demands to justify such huge chip areas. CraterLake (472.3mm²), BTS (373.6mm²), and also ARK (418.3mm²) all employ massive amounts of on-chip memory, consequently utilizing much larger chip areas compared to F1 (151.4mm²). An increase in fabrication cost, which is superlinear to the chip area [45], might dilute the benefits of using larger designs despite the also superlinear increase in performance. Multi-chip modules [70], [74] and 3D integration [23], [24], as used in recent AMD CPUs [19] with stacked SRAM memories [95], are promising solutions that can lower the fabrication cost by dividing monolithic FHE accelerator designs into chiplet designs. It is our future work to explore such chiplet FHE accelerator designs.

IX. RELATED WORK

CPU/GPU acceleration of HE: Prior works have tried to accelerate HE by better use of general-purpose processors: CPUs and GPUs. [56] and [15] accelerate HE using CPU SIMD extensions. Meanwhile, Lattigo [38] exploits the latest algorithmic optimizations [16], [42] to accelerate HE ops. However, the relatively low computational power of CPUs has resulted in limited success. To overcome the limitations of CPUs, prior works [3]–[5], [55] have attempted to accelerate HE by utilizing the high parallel computing power of

Table VII
COMPARISON BETWEEN ARK AND RECENT FHE ACCELERATORS, CRATERLAKE [88] AND BTS [59]

	ARK	CraterLake	BTS
Technology	7nm	12/14nm	7nm
Word size	64-bit	28-bit	64-bit
On-chip memory	512MB	256MB	512MB
$T_{A.S.}$	14.3ns	17.6ns	45.4ns
HELR	7.42ms	15.2ms	28.4ms
ResNet-20	0.125s	0.321s	1.91s
Sorting	1.99s	-	15.6s
Area	418.3mm ²	472.3mm ²	373.6mm ²
Peak power	281.3W	> 317W	163.2W

GPUs. However, most prior GPU works [3]–[5] accelerate part of primitive HE ops or do not include acceleration of bootstrapping. 100x [55] accelerates all HE ops including CKKS bootstrapping. 100x mitigates the off-chip memory bottleneck in a GPU by performing kernel fusion, which leads to $242\times$ faster bootstrapping over a CPU. However, the limited on-chip storage capacity of the GPU hinders kernel fusion for some functions [58]. Yet there are other GPU acceleration works [67], [73], [76], [93] targeting boolean HE schemes [29], [36].

FPGA designs for HE: Prior works have implemented hardware logic for HE ops using FPGAs [2], [60], [61], [83], [85]. However, most of them either do not cover entire HE ops [60], [61] or target LHE [81], [83], [85]. HEAX [83] targets CKKS and accelerates HMult with specialized NTT cores and deep pipelines. However, it only supports non-bootstrappable, limited parameters. FAB [2] is the first FPGA work to accelerate CKKS bootstrapping with a carefully designed datapath for key-switching that minimizes the working set to fully leverage 43MB of its on-chip storage. FAB shows comparable performance to GPUs [55] and ASIC designs [59], [87] for some FHE workloads, but its bootstrapping performance is still an order of magnitude lower than recent ASIC FHE accelerators [59], [88] including ARK.

ASIC HE accelerators: A number of ASIC HE accelerators [59], [81], [87], [88] have been proposed. Cheetah [81] targets privacy-preserving ML using LHE, based on the implementation of Gazelle [57], and designs an ASIC accelerator for such workloads. Gazelle performs ML inference by combining LHE and multi-party computation (MPC) [96]. Such implementation requires frequent server-client communication and needs re-encryption to support complex workloads inducing expensive network overheads. F1 [87] is the first ASIC work that reports the performance of CKKS bootstrapping. By using 64MB of on-chip memory and high throughput FUs, including NTTU, F1 improves the performance of primitive HE ops; however, it does not target the parameters suitable for practical bootstrapping. Recently

proposed BTS [59] and CraterLake [88] are state-of-the-art ASIC FHE accelerators that fully consider appropriate parameters for CKKS bootstrapping.

X. CONCLUSION

In this paper, we present a set of fabrication-technology-aware algorithmic optimizations and hardware design for fully homomorphic encryption (FHE). To mitigate the off-chip memory bandwidth bottleneck of bootstrapping, we present the following key algorithmic enhancements: *minimum key-switching* and *on-the-fly limb extension*. These two optimizations significantly reduce off-chip memory access, fully reaping the high computational capabilities of domain-specific architectures. We propose ARK, an accelerator addressing new computation and data movement constraints of FHE through an efficient functional unit (FU) microarchitecture that minimizes on-chip data movement. ARK features a data-access-pattern-aware data distribution policy and a memory organization simplifying dataflow, enabling streamlined execution across the FUs. Overall, ARK provides two to four orders of magnitude higher performance than prior works. This enables ARK to provide real-time encrypted CNN inference, showing 0.125 seconds of inference time for the ResNet-20 model.

ACKNOWLEDGMENT

The authors thank Jaewan Choi, the anonymous reviewers, and the shepherd for their constructive comments. This work was supported by the National Research Foundation of Korea (NRF) grant (No. 2020R1A2C2010601) and Institute of Information & communications Technology Planning & Evaluation (IITP) grants (No. 2020-0-00840, and No. 2021-0-01343) funded by the Korean government (MSIT). The EDA tool was supported by the IC Design Education Center (IDEC), Korea. Jongmin Kim is with the Interdisciplinary Program in Artificial Intelligence, Seoul National University (SNU). Sangpyo Kim is with the Department of Intelligence and Information, SNU. Jung Ho Ahn, the corresponding author, is with the Department of Intelligence and Information, the Interdisciplinary Program in Artificial Intelligence, and the Research Institute for Convergence Science, SNU.

REFERENCES

- [1] D. Abts, J. Ross, J. Sparling, M. Wong-VanHaren, M. Baker, T. Hawkins, A. Bell, J. Thompson, T. Kahsai, G. Kimmell, J. Hwang, R. Leslie-Hurd, M. Bye, E. Creswick, M. Boyd, M. Venigalla, E. Laforge, J. Purdy, P. Kamath, D. Maheshwari, M. Beidler, G. Rosseel, O. Ahmad, G. Gagarin, R. Czekalski, A. Rane, S. Parmar, J. Werner, J. Sproch, A. Macias, and B. Kurtz, "Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads," in *ISCA*, 2020.
- [2] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "FAB: An FPGA-based Accelerator for Bootstrappable Fully Homomorphic Encryption," *arXiv preprint arXiv:2207.11872*, 2022.
- [3] A. Al Badawi, L. Hoang, C. F. Mun, K. Laine, and K. M. M. Aung, "Privft: Private and Fast Text Classification with Homomorphic Encryption," *IEEE Access*, vol. 8, pp. 226 544–226 556, 2020.
- [4] A. Al Badawi, Y. Polyakov, K. M. M. Aung, B. Veeravalli, and K. Rohloff, "Implementation and Performance Evaluation of RNS Variants of the BFV Homomorphic Encryption Scheme," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 2, pp. 941–956, 2019.
- [5] A. Al Badawi, B. Veeravalli, C. F. Mun, and K. M. M. Aung, "High-Performance FV Somewhat Homomorphic Encryption on GPUs: An Implementation Using CUDA," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 2, pp. 143–163, 2018.
- [6] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter, S. Lokam, D. Micciancio, D. Moody, T. Morrison, A. Sahai, and V. Vaikuntanathan, "Homomorphic Encryption Standard," in *Protecting Privacy through Homomorphic Encryption*. Springer, 2021, pp. 31–62.
- [7] F. Armknecht, C. Boyd, C. Carr, K. Gjøsteen, A. Jäschke, C. A. Reuter, and M. Strand, "A Guide to Fully Homomorphic Encryption," *IACR Cryptology ePrint Archive*, no. 1192, 2015.
- [8] K. Asanovic, "Vector Microprocessors," Ph.D. dissertation, University of California, Berkeley, 1998.
- [9] C. Auth, A. Aliyarunju, M. Asoro, D. Bergstrom, V. Bhagwat, J. Birdsall, N. Bisnik, M. Buehler, V. Chikarmane, G. Ding, Q. Fu, H. Gomez, W. Han, D. Hanken, M. Haran, M. Hattendorf, R. Heussner, H. Hiramatsu, B. Ho, S. Jaloviar, I. Jin, S. Joshi, S. Kirby, S. Kosaraju, H. Kothari, G. Leatherman, K. Lee, J. Leib, A. Madahavan, K. Marla, H. Meyer, T. Mule, C. Parker, S. Parthasarathy, C. Pelto, L. Pipes, I. Post, M. Prince, A. Rahman, S. Rajamani, A. Saha, J. Dacuna Santos, M. Sharma, V. Sharma, J. Shin, P. Sinha, P. Smith, M. Sprinkle, A. St. Amour, C. Staus, R. Suri, D. Towner, A. Tripathi, A. Tura, C. Ward, and A. Yeoh, "A 10nm High Performance and Low-Power CMOS Technology Featuring 3rd Generation FinFET Transistors, Self-Aligned Quad Patterning, Contact over Active Gate and Cobalt Local Interconnects," in *IEEE International Electron Devices Meeting*, 2017.
- [10] D. H. Bailey, "FFTs in External or Hierarchical Memory," *Journal of Supercomputing*, vol. 4, no. 1, pp. 23–35, 1990.
- [11] J. Bajard, J. Eynard, M. A. Hasan, and V. Zucca, "A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes," in *Selected Areas in Cryptography*, 2016.
- [12] K. Banerjee and A. Mehrotra, "A Power-Optimal Repeater Insertion Methodology for Global Interconnects in Nanometer Designs," *IEEE Transactions on Electron Devices*, vol. 49, no. 11, pp. 2001–2007, 2002.
- [13] P. Barrett, "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *Annual International Conference on the Theory and Application of Cryptographic Techniques*, 1986.
- [14] J. Birmes, K. Asanovic, C.-W. Chin, and J. Demmel, "Optimizing Matrix Multiply using PHiPAC: a Portable, High-Performance, ANSI C Coding Methodology," in *International Conference on Supercomputing*, 1997, pp. 253–260.

- [15] F. Boemer, S. Kim, G. Seifu, F. D. M. de Souza, and V. Gopal, "Intel HEXL: Accelerating Homomorphic Encryption with Intel AVX512-IFMA52," in *Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2021.
- [16] J. Bossuat, C. Mouchet, J. R. Troncoso-Pastoriza, and J. Hubaux, "Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2021.
- [17] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) Fully Homomorphic Encryption without Bootstrapping," *ACM Transactions on Computing Theory*, vol. 6, no. 3, 2014.
- [18] Z. Brakerski and V. Vaikuntanathan, "Efficient Fully Homomorphic Encryption from (Standard) LWE," *SIAM Journal on Computing*, vol. 43, no. 2, pp. 831–871, 2014.
- [19] T. Burd, W. Li, J. Pistole, S. Venkataraman, M. McCabe, T. Johnson, J. Vinh, T. Yiu, M. Wasio, H.-H. Wong, D. Lieu, J. White, B. Munger, J. Lindner, J. Olson, S. Bakke, J. Sniderman, C. Henrion, R. Schreiber, E. Busta, B. Johnson, T. Jackson, A. Miller, R. Miller, M. Pickett, A. Horiuchi, J. Dvorak, S. Balagangadharan, S. Ammikkallangal, and P. Kumar, "Zen3: The AMD 2nd-Generation 7nm x86-64 Microprocessor Core," in *IEEE International Solid-State Circuits Conference*, 2022.
- [20] J. Chang, Y. Chen, G. Chan, H. Cheng, P. Wang, Y. Lin, H. Fujiwara, R. Lee, H. Liao, P. Wang, G. Yeap, and Q. Li, "15.1 A 5nm 135Mb SRAM in EUV and High-Mobility-Channel FinFET Technology with Metal Coupling and Charge-Sharing Write-Assist Circuitry Schemes for High-Density and Low-VMIN Applications," in *IEEE International Solid-State Circuits Conference*, 2020.
- [21] J. Chang, Y. Chen, W. Chan, S. P. Singh, H. Cheng, H. Fujiwara, J. Lin, K. Lin, J. Hung, R. Lee, H. Liao, J. Liaw, Q. Li, C. Lin, M. Chiang, and S. Wu, "12.1 A 7nm 256Mb SRAM in High-K Metal-Gate FinFET Technology with Write-Assist Circuitry for Low-VMIN Applications," in *IEEE International Solid-State Circuits Conference*, 2017.
- [22] H. Chen and K. Han, "Homomorphic Lower Digits Removal and Improved FHE Bootstrapping," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2018.
- [23] M. F. Chen, C. S. Lin, E. B. Liao, W. C. Chiou, C. C. Kuo, C. C. Hu, C. H. Tsai, C. T. Wang, and D. Yu, "SoIC for Low-Temperature, Multi-Layer 3D Memory Integration," in *IEEE Electronic Components and Technology Conference*, 2020.
- [24] M.-F. Chen, F.-C. Chen, W.-C. Chiou, and C. Doug, "System on Integrated Chips (SoIC (TM) for 3D Heterogeneous Integration," in *IEEE Electronic Components and Technology Conference*, 2019.
- [25] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A Full RNS Variant of Approximate Homomorphic Encryption," in *Selected Areas in Cryptography*, 2018.
- [26] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for Approximate Homomorphic Encryption," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2018.
- [27] J. H. Cheon, A. Kim, M. Kim, and Y. S. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *International Conference on the Theory and Applications of Cryptology and Information Security*, 2017.
- [28] J. H. Cheon, Y. Son, and D. Yhee, "Practical FHE Parameters against Lattice Attacks," *Journal of the Korean Mathematical Society*, vol. 59, no. 1, pp. 35–51, 2022.
- [29] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast Fully Homomorphic Encryption Over the Torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [30] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "NVIDIA A100 Tensor Core GPU: Performance and Innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.
- [31] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm FinFET Predictive Process Design Kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [32] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [33] B. R. Curtis and R. Player, "On the Feasibility and Impact of Standardising Sparse-secret LWE Parameter Sets for Homomorphic Encryption," in *ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2019.
- [34] L. de Castro, R. Agrawal, R. T. Yazicigil, A. P. Chandrakasan, V. Vaikuntanathan, C. Juvekar, and A. Joshi, "Does Fully Homomorphic Encryption Need Compute Acceleration?" *arXiv preprint arXiv:2112.06396*, 2021.
- [35] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2009.
- [36] L. Ducas and D. Micciancio, "FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2015.
- [37] A. C. Elster and T. A. Haugdahl, "Nvidia Hopper GPU and Grace CPU Highlights," *Computing in Science & Engineering*, vol. 24, no. 2, pp. 95–100, 2022.
- [38] EPFL-LDS and Tune Insight SA, "Lattigo v3.0.1," Feb 2022. [Online]. Available: <https://github.com/tuneinsight/lattigo>
- [39] J. Fan and F. Vercauteren, "Somewhat Practical Fully Homomorphic Encryption," *IACR Cryptology ePrint Archive*, no. 144, 2012.
- [40] W. M. Gentleman and G. Sande, "Fast Fourier Transforms: For Fun and Profit," in *AFIPS Fall Joint Computer Conference*, 1966.
- [41] C. Gentry, "Fully Homomorphic Encryption Using Ideal Lattices," in *ACM Symposium on Theory of Computing*, 2009.
- [42] S. Halevi and V. Shoup, "Faster Homomorphic Linear Transformations in HELib," in *Annual International Cryptology Conference*, 2018.
- [43] K. Han, S. Hong, J. H. Cheon, and D. Park, "Logistic Regression on Homomorphic Encrypted Data at Scale," in *AAAI Conference on Artificial Intelligence*, 2019.
- [44] K. Han and D. Ki, "Better Bootstrapping for Approximate Homomorphic Encryption," in *Cryptographers' Track at the RSA Conference*, 2020.
- [45] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [46] R. Ho, K. Mai, and M. Horowitz, "The Future of Wires," *Proceedings of the IEEE*, vol. 89, no. 4, pp. 490–504, 2001.

- [47] S. Hong, S. Kim, J. Choi, Y. Lee, and J. H. Cheon, "Efficient Sorting of Homomorphic Encrypted Data With k-Way Sorting Network," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4389–4404, 2021.
- [48] M. Horowitz, "1.1 Computing's Energy Problem (and what we can do about it)," in *IEEE International Conference on Solid-State Circuits Conference*, 2014, pp. 10–14.
- [49] IEEE, "International Roadmap for Devices and Systems: 2018," Tech. Rep., 2018. [Online]. Available: <https://irds.ieee.org/editions/2018/>
- [50] J. Janssen and H. Corporaal, "Partitioned Register File for TTAs," in *MICRO*, 1995.
- [51] JEDEC, "High Bandwidth Memory (HBM) DRAM," Tech. Rep. JESD235D, 2021.
- [52] JEDEC, "High Bandwidth Memory DRAM (HBM3)," Tech. Rep. JESD238, 2022.
- [53] W. Jeong, S. Maeda, H. Lee, K. Lee, T. Lee, D. Park, B. Kim, J. Do, T. Fukai, D. Kwon, K. Nam, W. Rim, M. Jang, H. Kim, Y. Lee, J. Park, E. Lee, D. Ha, C. Park, H. Cho, S. Jung, and H. Kang, "True 7nm Platform Technology featuring Smallest FinFET and Smallest SRAM cell by EUV, Special Constructs and 3rd Generation Single Diffusion Break," in *IEEE Symposium on VLSI Technology*, 2018.
- [54] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. C. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. A. Patterson, "Ten Lessons From Three Generations Shaped Google's TPUv4i: Industrial Product," in *ISCA*, 2021.
- [55] W. Jung, S. Kim, J. Ahn, J. H. Cheon, and Y. Lee, "Over 100x Faster Bootstrapping in Fully Homomorphic Encryption through Memory-centric Optimization with GPUs," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 4, pp. 114–148, 2021.
- [56] W. Jung, E. Lee, S. Kim, J. Kim, N. Kim, K. Lee, C. Min, J. H. Cheon, and J. Ahn, "Accelerating Fully Homomorphic Encryption Through Architecture-Centric Analysis and Optimization," *IEEE Access*, vol. 9, pp. 98 772–98 789, 2021.
- [57] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A Low Latency Framework for Secure Neural Network Inference," in *USENIX Security Symposium*, 2018.
- [58] S. Kim, W. Jung, J. Park, and J. Ahn, "Accelerating Number Theoretic Transformations for Bootstrappable Homomorphic Encryption on GPUs," in *IEEE International Symposium on Workload Characterization*, 2020.
- [59] S. Kim, J. Kim, M. J. Kim, W. Jung, J. Kim, M. Rhu, and J. Ahn, "BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption," in *ISCA*, 2022.
- [60] S. Kim, K. Lee, W. Cho, J. H. Cheon, and R. A. Rutenbar, "FPGA-based Accelerators of Fully Pipelined Modular Multipliers for Homomorphic Encryption," in *International Conference on ReConfigurable Computing and FPGAs*, 2019.
- [61] S. Kim, K. Lee, W. Cho, Y. Nam, J. H. Cheon, and R. A. Rutenbar, "Hardware Architecture of a Number Theoretic Transform for a Bootstrappable RNS-based Homomorphic Encryption Scheme," in *IEEE International Symposium on Field-Programmable Custom Computing Machines*, 2020.
- [62] S. Knowles, "Graphcore," in *IEEE Hot Chips 33 Symposium*, 2021.
- [63] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, Tech. Rep., 2009.
- [64] E. Lee, J.-W. Lee, J. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and W. Choi, "Low-Complexity Deep Convolutional Neural Networks on Fully Homomorphic Encryption Using Multiplexed Parallel Convolutions," in *International Conference on Machine Learning*, 2022.
- [65] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim, and J.-S. No, "Privacy-Preserving Machine Learning With Fully Homomorphic Encryption for Deep Neural Network," *IEEE Access*, vol. 10, pp. 30 039–30 054, 2022.
- [66] J. Lee, E. Lee, Y. Lee, Y. Kim, and J. No, "High-Precision Bootstrapping of RNS-CKKS Homomorphic Encryption Using Optimal Minimax Polynomial Approximation and Inverse Sine Function," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2021.
- [67] M. S. Lee, Y. Lee, J. H. Cheon, and Y. Paek, "Accelerating Bootstrapping in FHEW using GPUs," in *IEEE International Conference on Application-specific Systems, Architectures and Processors*, 2015.
- [68] Y. Lee, J.-W. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and H. Kang, "High-Precision Bootstrapping for Approximate Homomorphic Encryption by Error Variance Minimization," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2022.
- [69] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, "Disaggregated Memory for Expansion and Sharing in Blade Servers," in *ISCA*, 2009.
- [70] M.-S. Lin, T.-C. Huang, C.-C. Tsai, K.-H. Tam, K. C.-H. Hsieh, C.-F. Chen, W.-H. Huang, C.-W. Hu, Y.-C. Chen, S. K. Goel, C.-M. Fu, S. Rusu, C.-C. Li, S.-Y. Yang, M. Wong, S.-C. Yang, and F. Lee, "A 7-nm 4-GHz Arm-Core-Based CoWoS Chiplet Design for High-Performance Computing," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 4, pp. 956–966, 2020.
- [71] P. L. Montgomery, "Modular Multiplication without Trial Division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [72] P. Moon, V. Chikarmane, K. Fischer, R. Grover, T. A. Ibrahim, D. Ingerly, K. J. Lee, C. Litteken, T. Mule, and S. Williams, "Process and Electrical Results for the On-die Interconnect Stack for Intel's 45nm Process Generation," *Intel Technology Journal*, vol. 12, no. 2, 2008.
- [73] T. Morshed, M. M. A. Aziz, and N. Mohammed, "CPU and GPU Accelerated Fully Homomorphic Encryption," in *IEEE International Symposium on Hardware Oriented Security and Trust*, 2020.
- [74] S. Naffziger, K. Lepak, M. Paraschou, and M. Subramony, "2.2 AMD Chiplet Architecture for High-Performance Server and Desktop Products," in *IEEE International Solid-State Circuits Conference*, 2020.
- [75] S. Narasimha, B. Jagannathan, A. Ogino, D. Jaeger, B. Greene, C. Sheraw, K. Zhao, B. Haran, U. Kwon, A. K. M. Mahalingam, B. Kannan, B. Morganfeld, J. Dechene, C. Radens, A. Tessier, A. Hassan, H. Narisetty, I. Ahsan, M. Aminpur, C. An, M. Aquilino, A. Arya, R. Augur, N. Baliga, R. Bhelkar, G. Biery, A. Blauberg, N. Botjemscaia, A. Bryant, L. Cao, V. Chauhan, M. Chen, L. Cheng, J. Choo, C. Christiansen, T. Chu, B. Cohen, R. Coleman, D. Conklin, S. Crown, A. da Silva, D. Dechene, G. Derderian, S. Deshpande, G. Dilliway, K. Donegan, M. Eller, Y. Fan,

- Q. Fang, A. Gassaria, R. Gauthier, S. Ghosh, G. Gifford, T. Gordon, M. Gribelyuk, G. Han, J. Han, K. Han, M. Hasan, J. Higman, J. Holt, L. Hu, L. Huang, C. Huang, T. Hung, Y. Jin, J. Johnson, S. Johnson, V. Joshi, M. Joshi, P. Justison, S. Kalaga, T. Kim, W. Kim, R. Krishnan, B. Krishnan, K. Anil, M. Kumar, J. Lee, R. Lee, J. Lemon, S. Liew, P. Lindo, M. Lingalugari, M. Lipinski, P. Liu, J. Liu, S. Lucarini, W. Ma, E. Maciejewski, S. Madiseti, A. Malinowski, J. Mehta, C. Meng, S. Mitra, C. Montgomery, H. Nayfeh, T. Nigam, G. Northrop, K. Onishi, C. Ordonio, M. Ozbek, R. Pal, S. Parihar, O. Patterson, E. Ramanathan, I. Ramirez, R. Ranjan, J. Sarad, V. Sardesai, S. Saudari, C. Schiller, B. Senapati, C. Serrau, N. Shah, T. Shen, H. Sheng, J. Shepard, Y. Shi, M. Silvestre, D. Singh, Z. Song, J. Sporre, P. Srinivasan, Z. Sun, A. Sutton, R. Sweeney, K. Tabakman, M. Tan, X. Wang, E. Woodard, G. Xu, D. Xu, T. Xuan, Y. Yan, J. Yang, K. Yeap, M. Yu, A. Zainuddin, J. Zeng, K. Zhang, M. Zhao, Y. Zhong, R. Carter, C. Lin, S. Grunow, C. Child, M. Lagus, R. Fox, E. Kaste, G. Gomba, S. Samavedam, P. Agnello, and D. K. Sohn, "A 7nm CMOS Technology Platform for Mobile and High Performance Compute Application," in *IEEE International Electron Devices Meeting*, 2017.
- [76] NuCypher, "A GPU Implementation of Fully Homomorphic Encryption on Torus," Mar 2020. [Online]. Available: <https://github.com/nucypher/nufhe>
- [77] NVIDIA Corporation, "NVIDIA Tesla V100 GPU Architecture," Tech. Rep. WP-08608-001_v1.1, 2017. [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>
- [78] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems," in *MICRO*, 2017.
- [79] PALISADE Project, "PALISADE Lattice Cryptography Library (release 1.11.5)," Sep 2021. [Online]. Available: <https://palisade-crypto.org/>
- [80] R. Prabhakar and S. Jairath, "SambaNova SN10 RDU: Accelerating Software 2.0 with Dataflow," in *IEEE Hot Chips 33 Symposium*, 2021.
- [81] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, "Cheetah: Optimizing and Accelerating Homomorphic Encryption for Private Inference," in *HPCA*, 2021.
- [82] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," *Journal of the ACM*, vol. 56, no. 6, 2009.
- [83] M. S. Riazzi, K. Laine, B. Pelton, and W. Dai, "HEAX: An Architecture for Computing on Encrypted Data," in *ASPLOS*, 2020.
- [84] S. Rixner, W. Dally, B. Khailany, P. Mattson, U. Kapasi, and J. Owens, "Register Organization for Media Processing," in *HPCA*, 2000.
- [85] S. S. Roy, F. Turan, K. Järvinen, F. Vercauteren, and I. Verbauwhede, "FPGA-Based High-Performance Parallel Architecture for Homomorphic Computing on Encrypted Data," in *HPCA*, 2019.
- [86] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "SCALE-Sim: Systolic CNN Accelerator Simulator," *arXiv preprint arXiv:1811.02883*, 2018.
- [87] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, "F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption," in *MICRO*, 2021.
- [88] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, "CraterLake: A Hardware Accelerator for Efficient Unbounded Computation on Encrypted Data," in *ISCA*, 2022.
- [89] A. Shafaei, Y. Wang, X. Lin, and M. Pedram, "FinCACTI: Architectural Analysis and Modeling of Caches with Deeply-Scaled FinFET Devices," in *IEEE Computer Society Annual Symposium on VLSI*, 2014.
- [90] T. Song, J. Jung, W. Rim, H. Kim, Y. Kim, C. Park, J. Do, S. Park, S. Cho, H. Jung, B. Kwon, H. Choi, J. Choi, and J. S. Yoon, "A 7nm FinFET SRAM Using EUV Lithography with Dual Write-Driver-Assist Circuitry for Low-Voltage Applications," in *IEEE International Solid-State Circuits Conference*, 2018.
- [91] Tesla, "Tesla AI Day," 2021. [Online]. Available: <https://youtu.be/j0z4FweCy4M>
- [92] S. Thoziyoor, J. Ahn, M. Monchiero, J. B. Brockman, and N. P. Jouppi, "A Comprehensive Memory Modeling Tool and Its Application to the Design and Analysis of Future Memory Hierarchies," in *ISCA*, 2008.
- [93] Vernam Group, "CUDA-Accelerated Fully Homomorphic Encryption Library," Feb 2019. [Online]. Available: <https://github.com/vernamlab/cuFHE>
- [94] S. Wu, C. Lin, M. Chiang, J. Liaw, J. Cheng, S. Yang, C. Tsai, P. Chen, T. Miyashita, C. Chang, V. Chang, K. Pan, J. Chen, Y. Mor, K. Lai, C. Liang, H. Chen, S. Chang, C. Lin, C. Hsieh, R. Tsui, C. Yao, C. Chen, R. Chen, C. Lee, H. Lin, C. Chang, K. Chen, M. Tsai, K. Chen, Y. Ku, and S. Jang, "A 7nm CMOS Platform Technology Featuring 4th Generation FinFET Transistors with a 0.027 μ m² High Density 6-T SRAM cell for Mobile SoC Applications," in *IEEE International Electron Devices Meeting*, 2016.
- [95] J. Wu, R. Agarwal, M. Ciraula, C. Dietz, B. Johnson, D. Johnson, R. Schreiber, R. Swaminathan, W. Walker, and S. Naffziger, "3D V-Cache: the Implementation of a Hybrid-Bonded 64MB Stacked Cache for a 7nm x86-64 CPU," in *IEEE International Solid-State Circuits Conference*, 2022.
- [96] A. C. Yao, "How to Generate and Exchange Secrets," in *IEEE Symposium on Foundations of Computer Science*, 1986.